

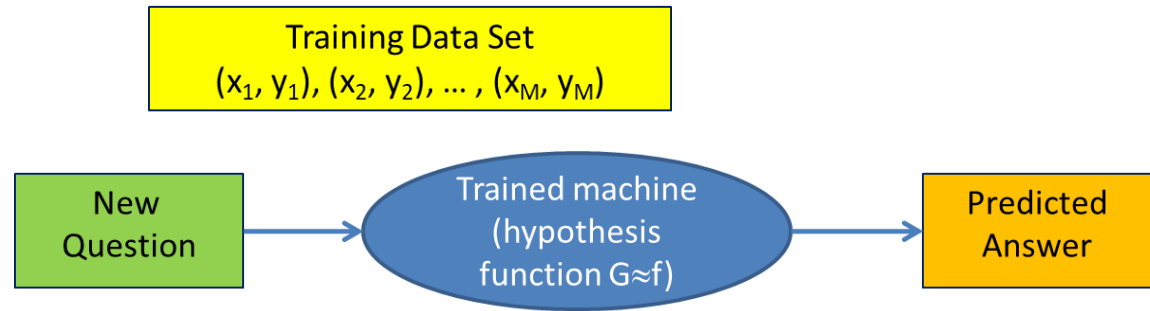
DASC7606

Deep Learning

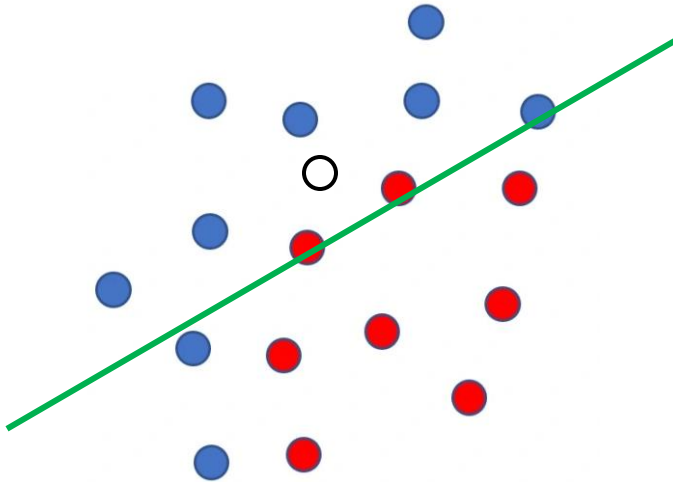
2. Neural Networks

Dr. Bethany Chan
Professor Francis Chin
2025/26

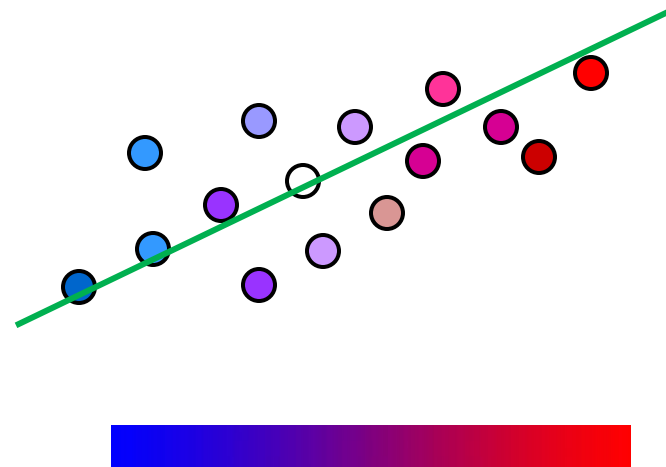
Linear models



Classification



Regression

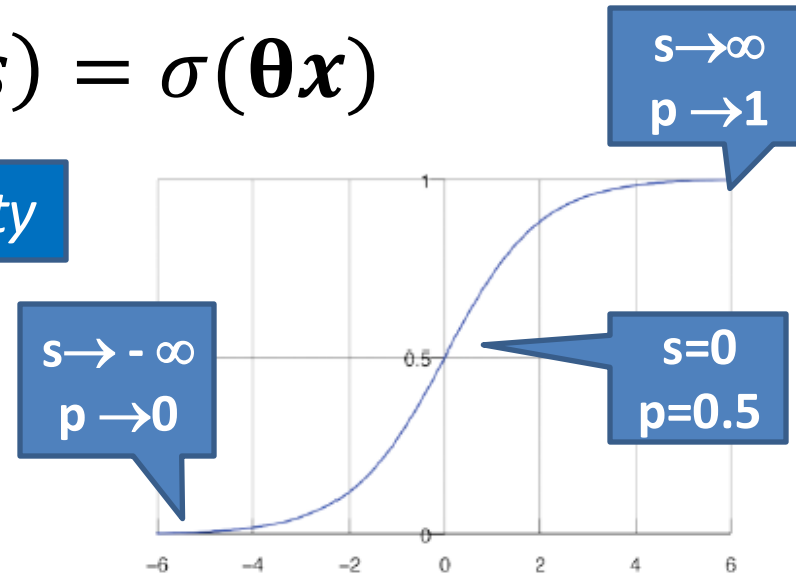
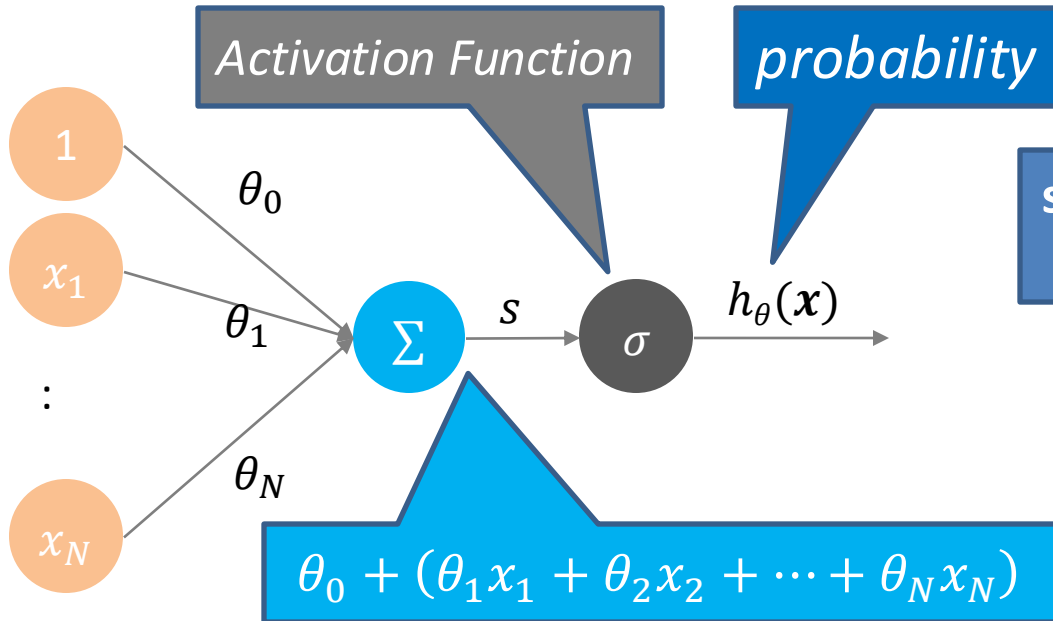


The dividing (classification) or approximate (regression) line/hyperplane is represented by $\theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_N x_N$

Learning coefficients or weights θ of a line or hyperplane to minimize error

Logistic Regression

Prediction : $h(\mathbf{x}) = \sigma(s) = \sigma(\boldsymbol{\theta}\mathbf{x})$



sigmoid $\sigma: \mathbb{R}^N \rightarrow [0, 1]$

Cost function (*cross entropy*)

$$J(h_{\theta}(\mathbf{x}), y) = -y \log(h_{\theta}(\mathbf{x})) - (1 - y) \log(1 - h_{\theta}(\mathbf{x}))$$

y = true answer

$y=1$

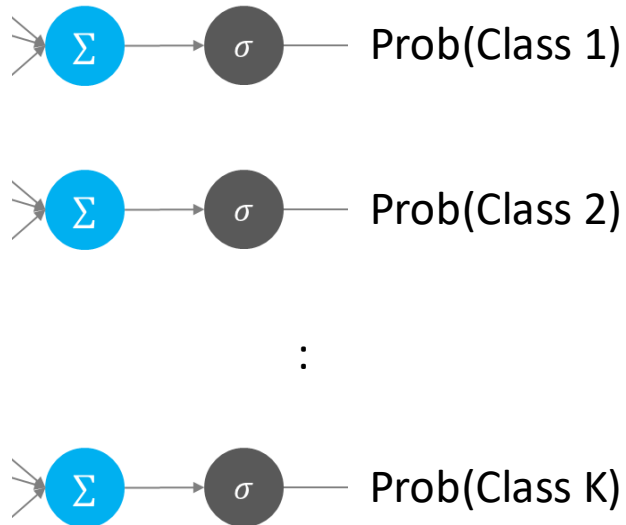
$y=0$

Multi-Classification: MNIST

MNIST (Mixed National Institute of Standards and Technology) database

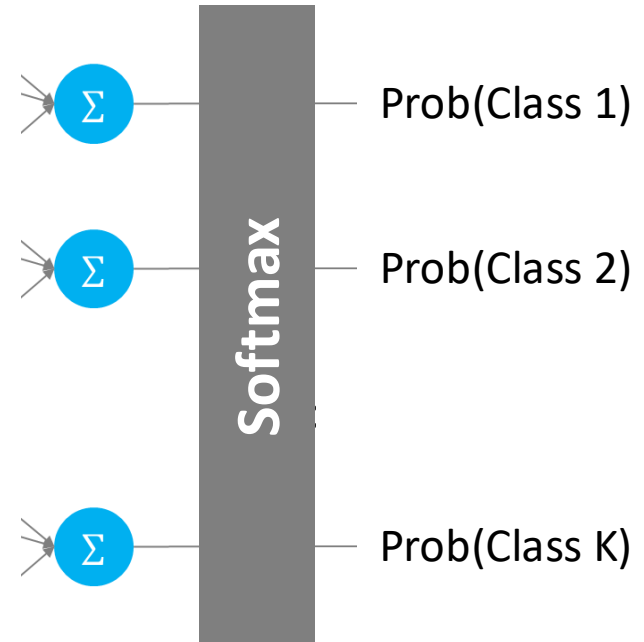


Multi-Classification: softmax



Prob(Class x) = Probability of Class x being the correct class

Class with **highest** probability is the predicted class



Softmax function: not only normalizes a set of scores to numbers in $[0,1]$ but also makes sure that the numbers all add up to 1

Multi-Classification: softmax vs sigmoid

For **sigmoid** (2-class):

$$\text{Class 1 probability: } \sigma(s) = \frac{1}{1+e^{-s}}$$

$$\text{Class 2 probability: } 1 - \sigma(s) = \frac{e^{-s}}{1+e^{-s}} = \frac{1}{1+e^s}$$

For **softmax** (K -class):

Given scores $s_1, s_2, \dots, s_K$

$$\text{Class } k \text{ probability: } \frac{e^{s_k}}{\sum e^{s_i}}$$

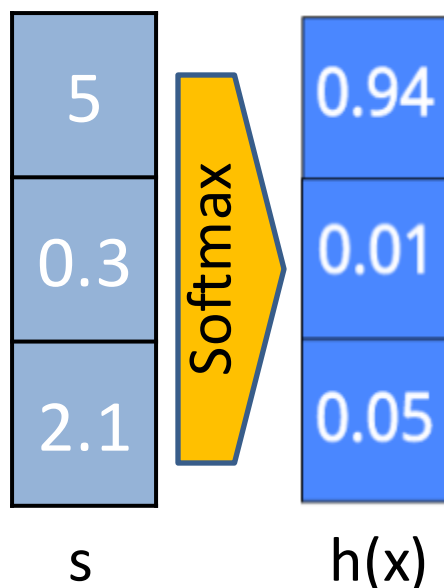
For **softmax** (2-class):

$$\text{Class 1 probability: } \frac{e^{s_1}}{e^{s_1}+e^{s_2}} = \frac{1}{1+e^{s_2-s_1}}$$

$$\text{Class 2 probability: } \frac{e^{s_2}}{e^{s_1}+e^{s_2}} = \frac{1}{1+e^{s_1-s_2}}$$

Multi-Classification: softmax

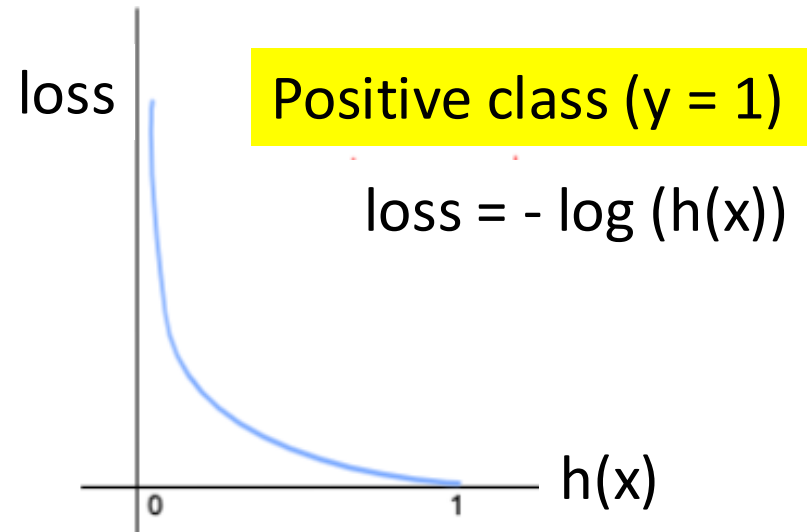
- **Question:** Why do we have to pass each value through an exponential before normalizing them? Why can't we just normalize the values themselves?
- **Answer:** This is because the goal of softmax is to make sure **one value is very high** (close to 1) and all **other values are very low** (close to 0).
- Softmax uses exponential to make sure this happens.



Multi-Classification: Loss Function

Categorical Cross Entropy

$$\begin{aligned} \text{Loss}(h(x), y) \\ = - \sum_c y \log(h(x)) \end{aligned}$$



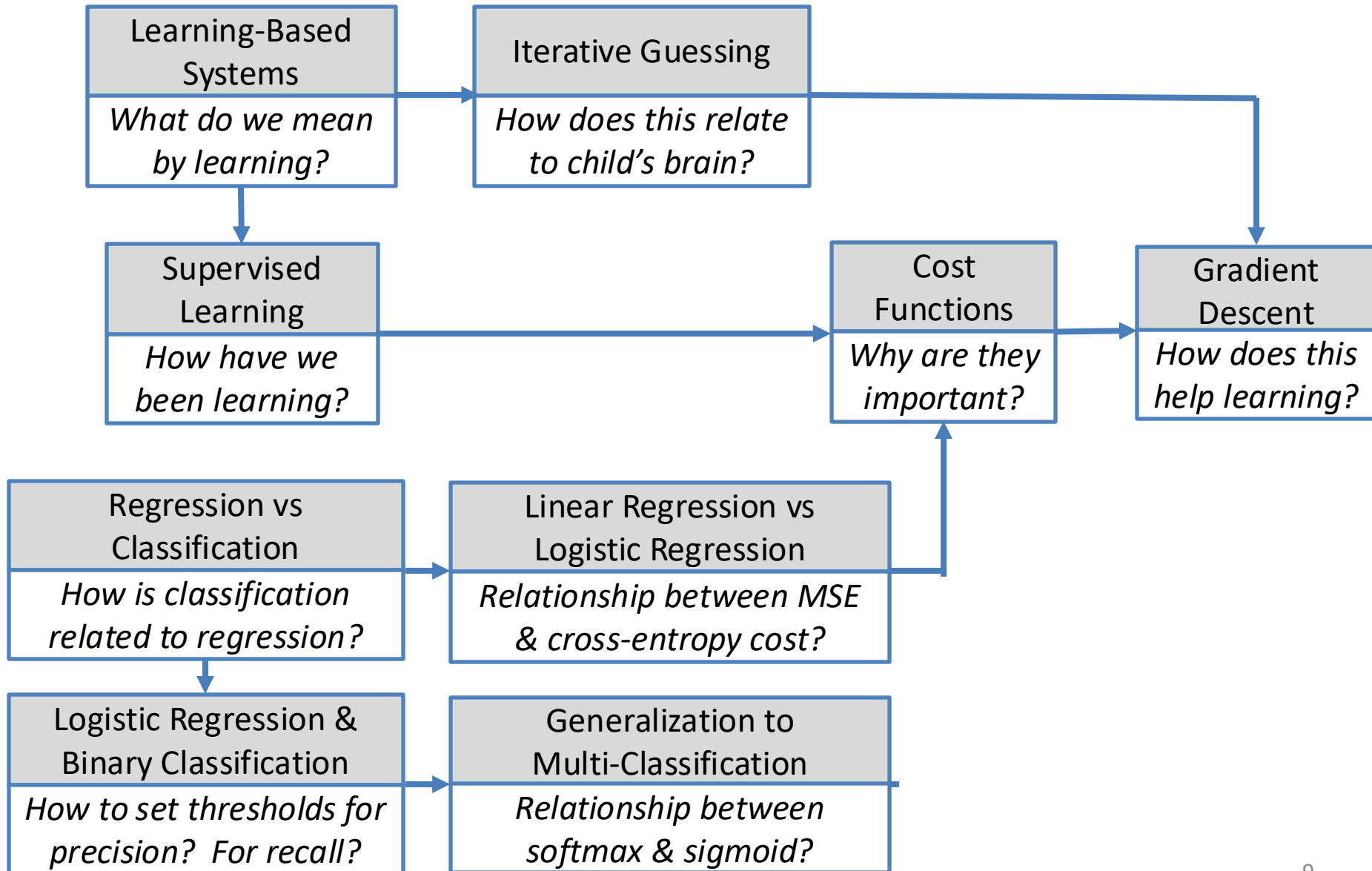
Ex:

Predicted $h(x)$	Label y
[0.94, 0.01, 0.05]	[0, 1, 0]

$$\text{Loss} = -[0 * \log(0.94) + 1 * \log(0.01) + 0 * \log(0.05)] = -\log(0.01)$$

The loss is large as the answer is wrong

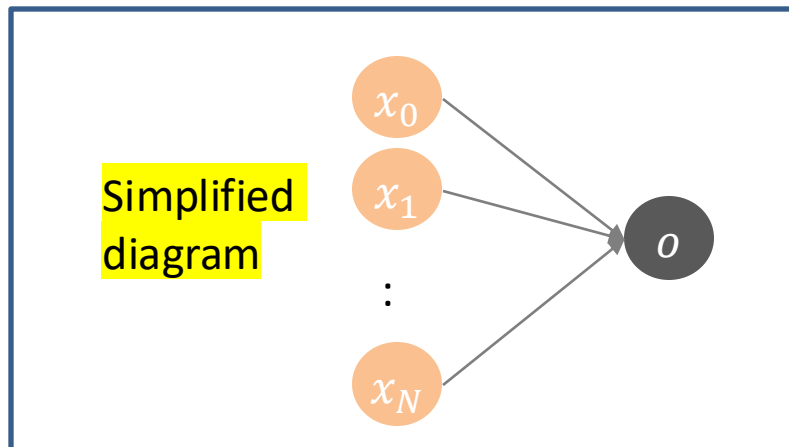
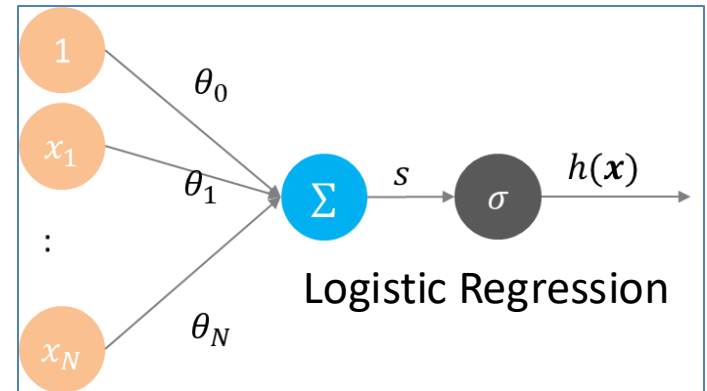
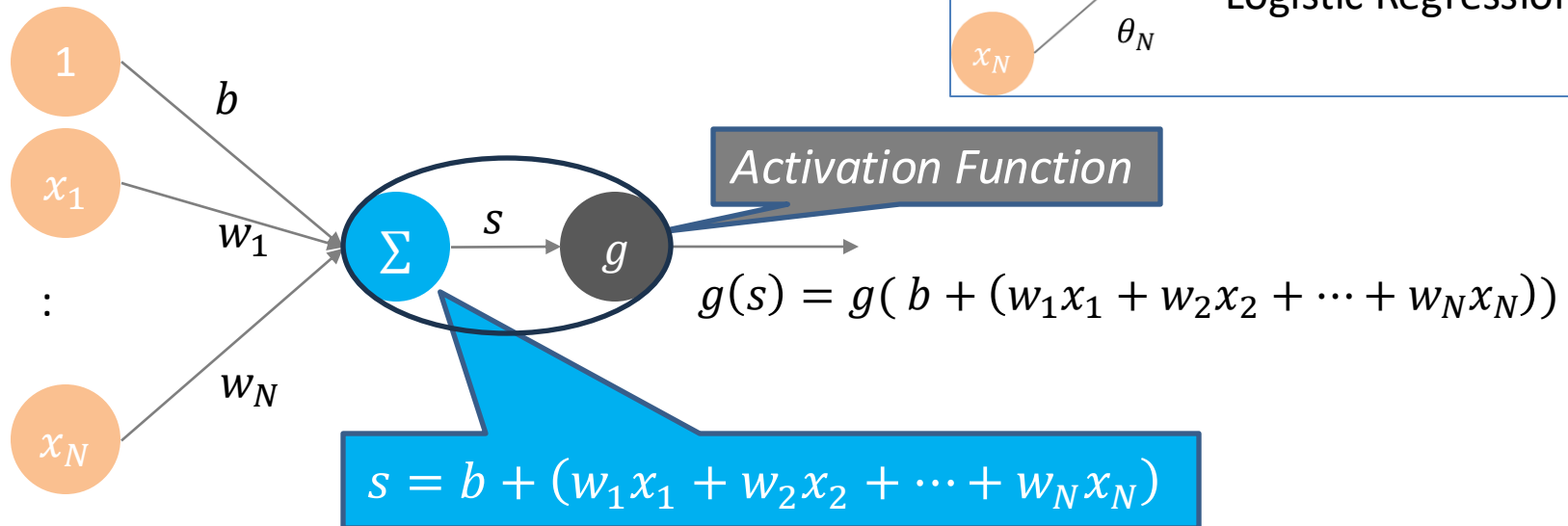
Perspectives – Mind Map (2)



NN Basics – Outline

1. The Perceptron
2. The Neural Network (NN)
3. Neural Networks vs Linear Models
4. NN and LOGIC functions
5. NN as a Function Approximator
6. Backpropagation (BP)
7. Framework for Training the NN
8. Example of Simple NN Classifier

1. The Perceptron



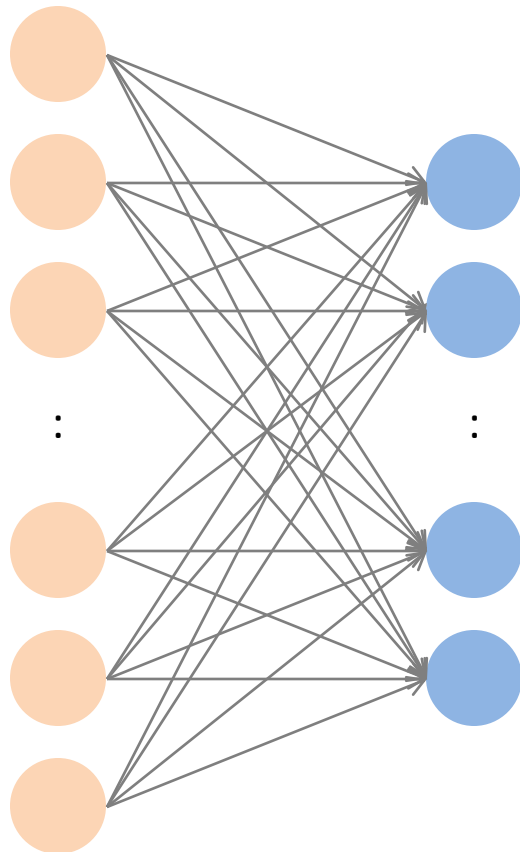
Notation:

- θ is now called w
- $\theta_0 = w_0 = b$ with $x_0=1$

The Perceptron: Multi-Output

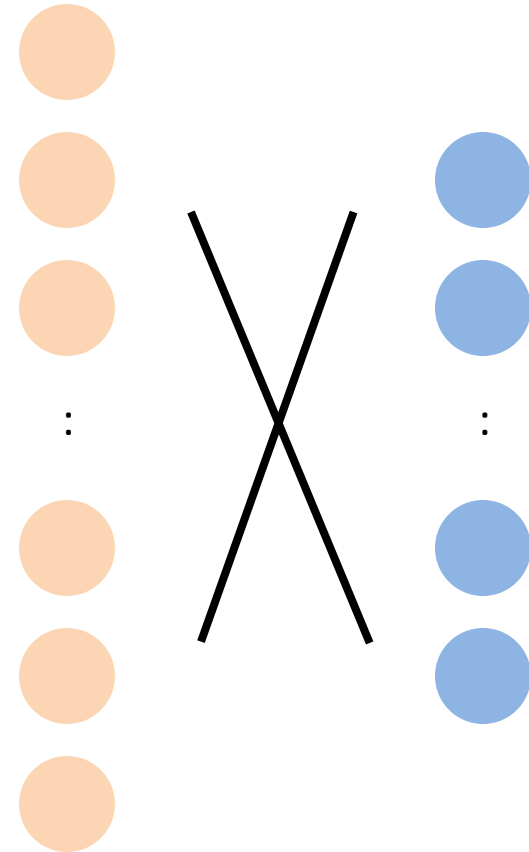
Input layer

Output layer



Input layer

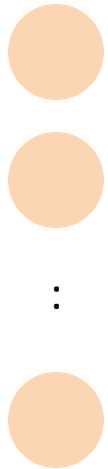
Output layer



2. The Neural Network (NN)

Multi-Layered Perceptron

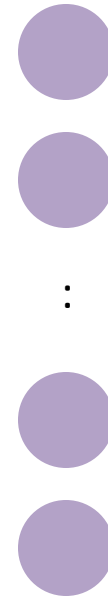
Input layer
(Layer 0)



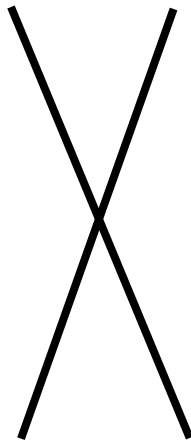
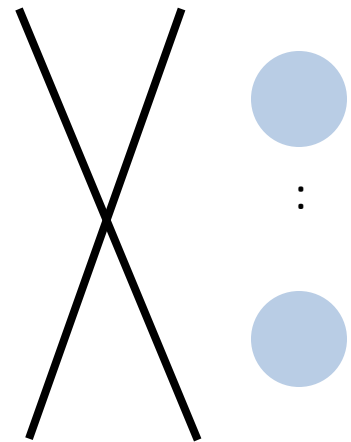
Hidden layer
(Layer 1)



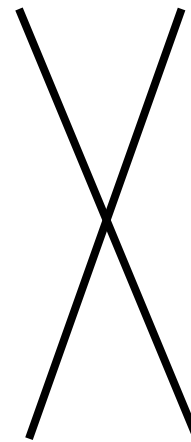
Hidden layer
(Layer L-1)



Output layer
(Layer L)

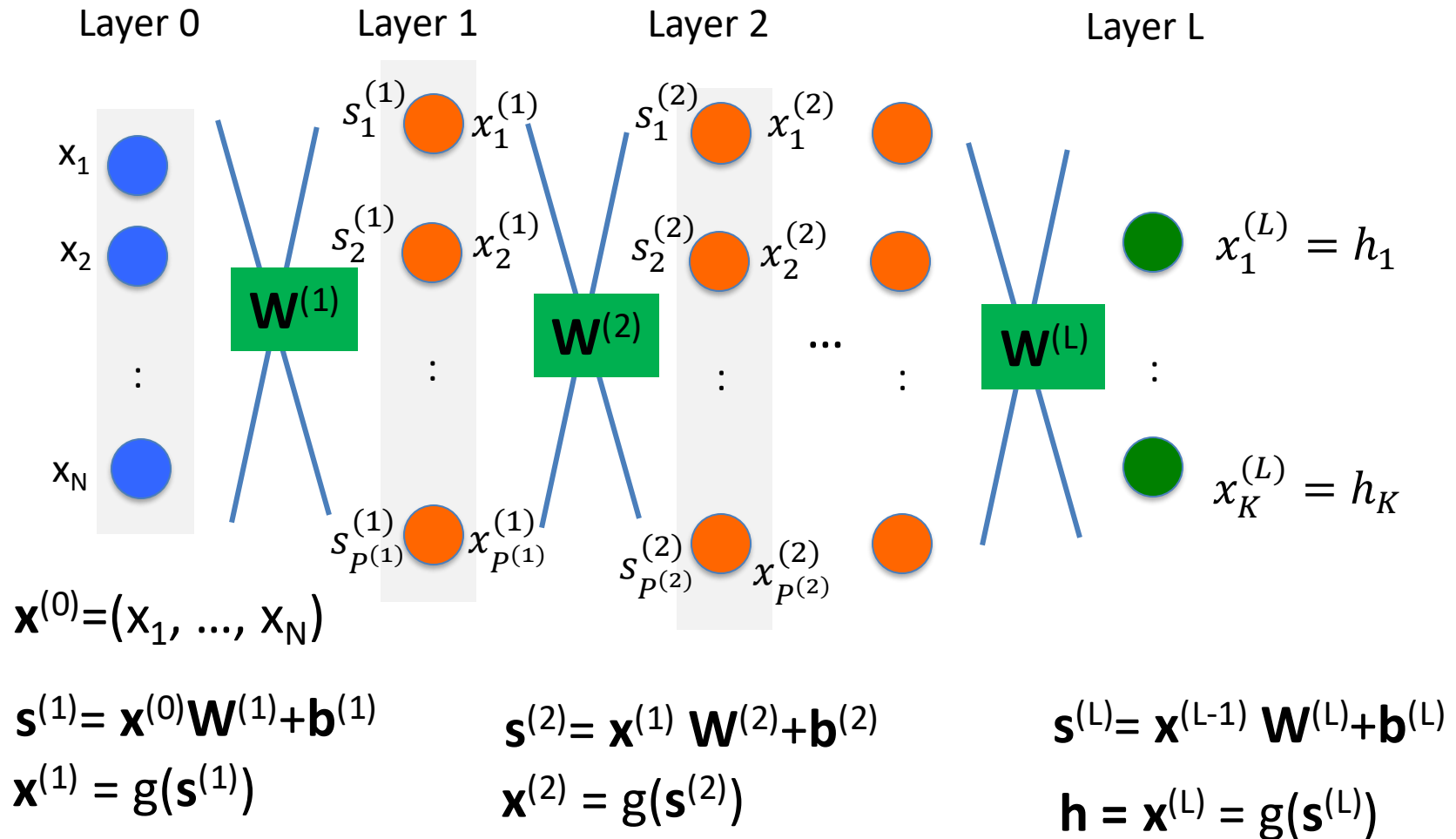


...



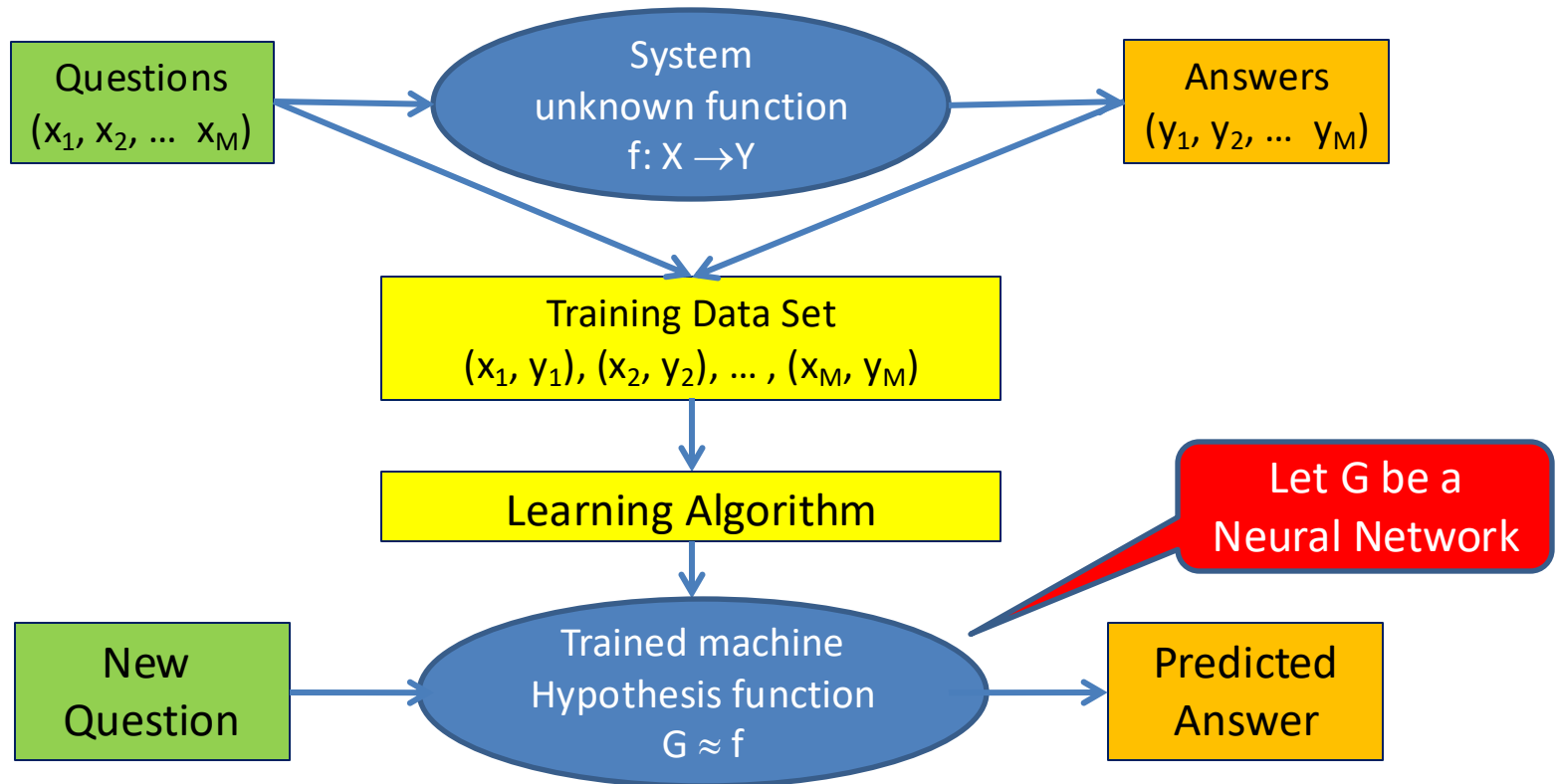
The Neural Network (NN)

Very structured program



3. Neural Networks vs Linear Models

Supervised Learning



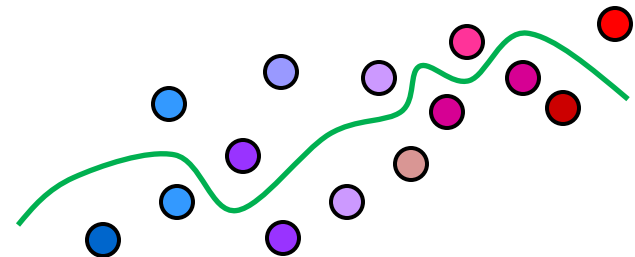
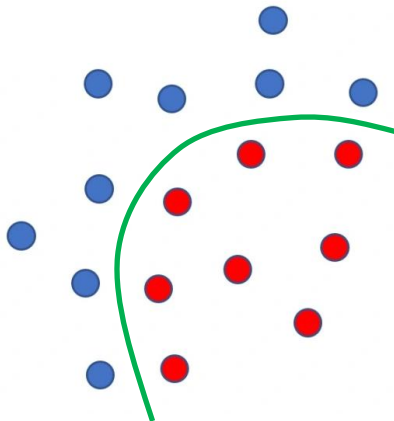
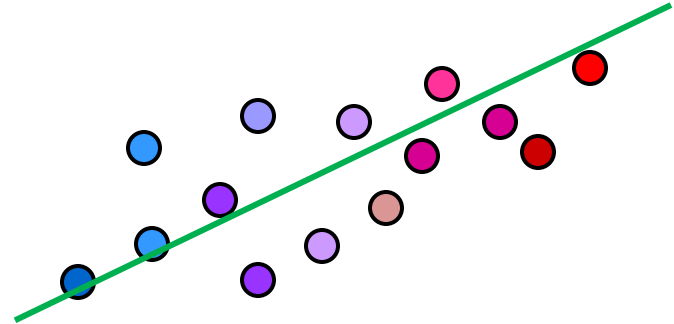
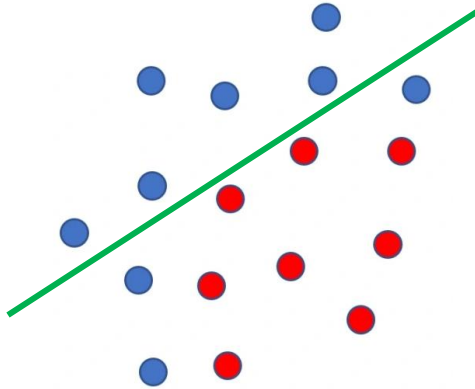
- Deep Learning uses NN (instead of linear models) for G. **Why?**
- **NN can approximate any function (to be shown)**

Neural Networks vs Linear Models

Classification

vs

Regression



Classification and logic functions

Classification: Yes or No, True or False, Approve or not, ...

Symptoms of heart attack:

chest pain ($x_1 = 1$) **AND** **shortness of breath** ($x_2 = 1$).

x_1 **AND** x_2

System triggers an alert:

sensor detects smoke ($x_1 = 1$)

OR

temperature is too high ($x_2 = 1$).

x_1 **OR** x_2

Approve the loan if the applicant has

both a **stable income** ($x_1 = 1$)

AND **good credit history** ($x_2 = 1$)

OR if the applicant has a **guarantor** ($x_3 = 1$).

(x_1 **AND** x_2) **OR** x_3

The model is trained
(by gradient descent)
for these applications.

Questions:

***Can NN be trained for
these applications?***

Answer:

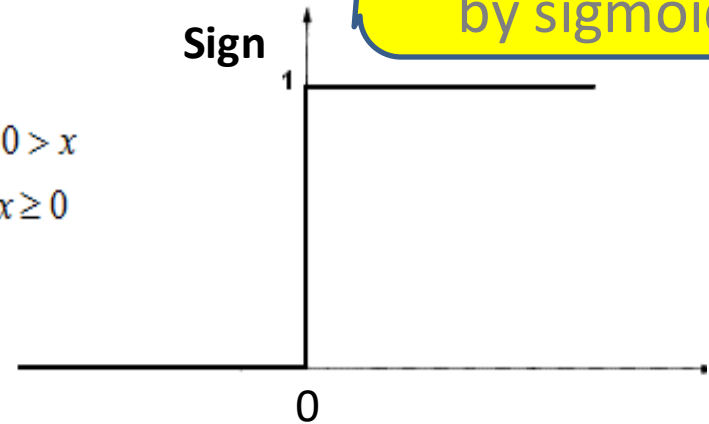
***Only if NN can perform
these logic functions.***

4. NN and LOGIC functions

Computing OR

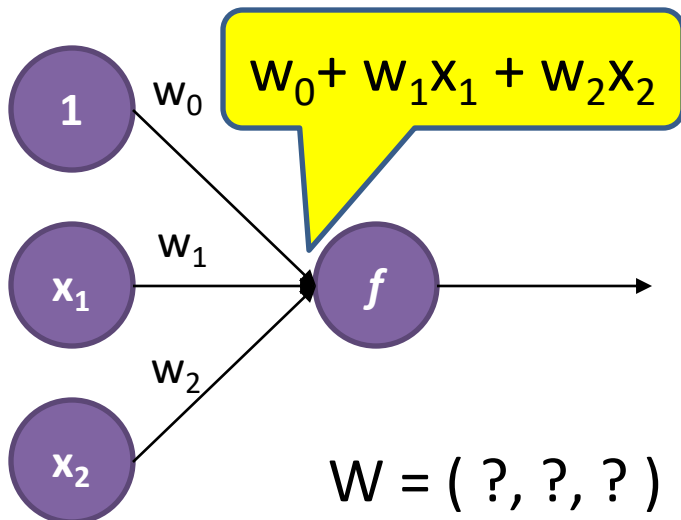
x_1	x_2	OR(x_1, x_2)
0	0	0
0	1	1
1	0	1
1	1	1

$$f(x) = \begin{cases} 0 & \text{if } 0 > x \\ 1 & \text{if } x \geq 0 \end{cases}$$



Activation function

Step function
- approximated
by sigmoid



$w_0 = ?$ when $x_1 = x_2 = 0$
 $w_0 < 0$, say $w_0 = -50$

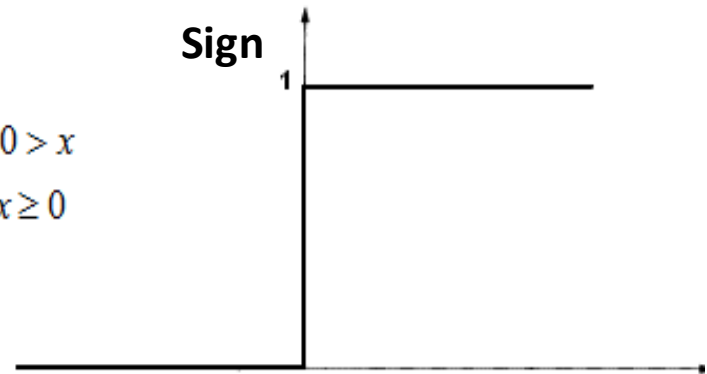
$w_1 = ?$ when $x_1 = 1, x_2 = 0$
 $w_1 + w_0 > 0$, say $w_1 = 100$

$w_2 = ?$ when $x_1 = 0, x_2 = 1$
 $w_2 + w_0 > 0$, say $w_2 = 100$

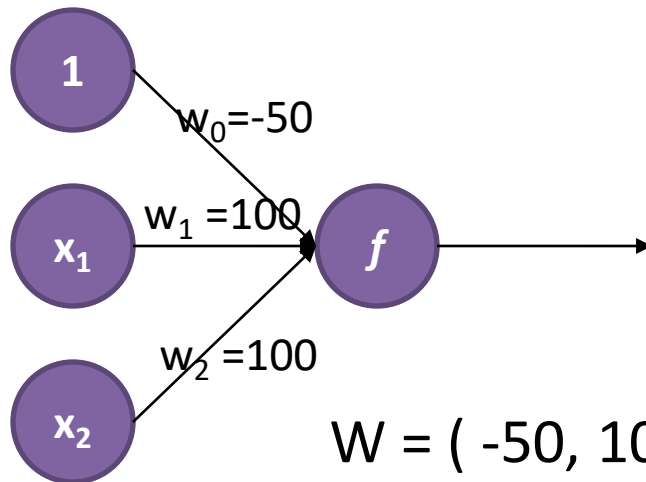
Computing OR

x_1	x_2	$\text{OR}(x_1, x_2)$
0	0	0
0	1	1
1	0	1
1	1	1

$$f(x) = \begin{cases} 0 & \text{if } 0 > x \\ 1 & \text{if } x \geq 0 \end{cases}$$



Activation function



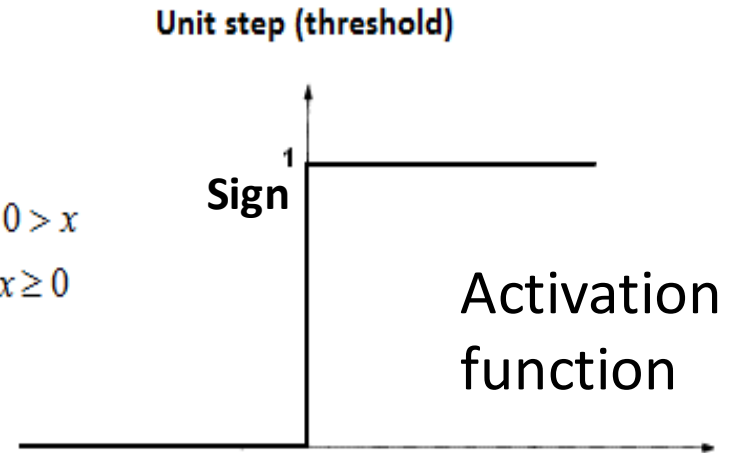
$$W = (-50, 100, 100)$$

x_1	x_2	$\text{OR}(x_1, x_2)$
0	0	0
0	1	1
1	0	1
1	1	1

Computing AND

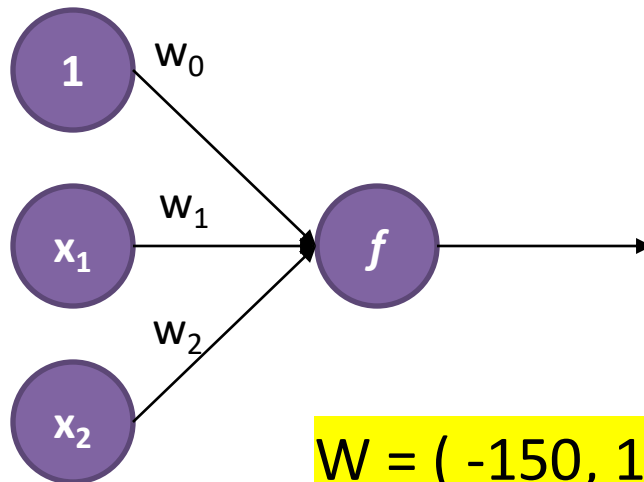
x_1	x_2	$\text{AND}(x_1, x_2)$
0	0	0
0	1	0
1	0	0
1	1	1

$$f(x) = \begin{cases} 0 & \text{if } 0 > x \\ 1 & \text{if } x \geq 0 \end{cases}$$



$w_0 = ?$

when $x_1 = x_2 = 0$
 $w_0 < 0$, say $w_0 = -150$



$w_1 = w_2 = w = ?$

when $x_1 = 1, x_2 = 0$
 or $x_1 = 0, x_2 = 1$
 $w + w_0 < 0$; $w < 150$

$W = (-150, 100, 100)$

when $x_1 = 1, x_2 = 1$
 $w_1 + w_2 + w_0 > 0$
 $2w > 150$
 Say $w = 100$

What is XOR ?

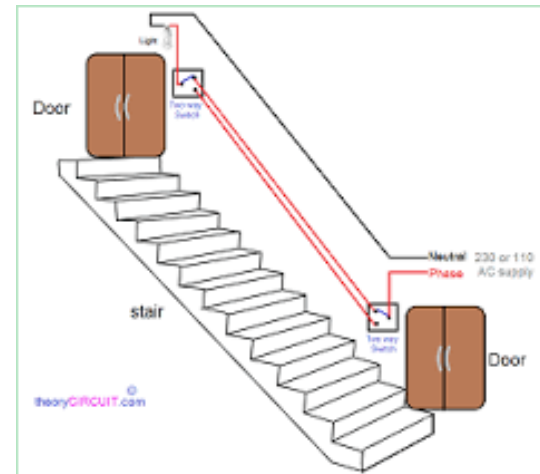
x_1	x_2	$x_1 \text{ XOR } x_2$
0	0	0
0	1	1
1	0	1
1	1	0

$$x_1 \oplus x_2$$



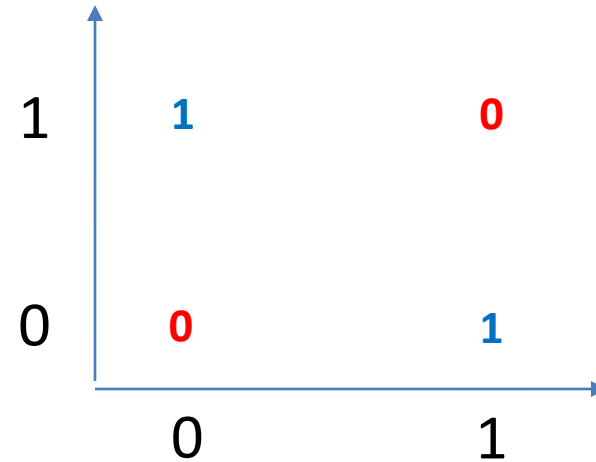
y – car can pass

- $y = x_1 \oplus x_2 = 1$ iff either x_1 or x_2 is 1, but not both
- Real world examples:
 - 2 cars with a narrow road
 - 2-way switch
 - Parity bit
(one 1 – odd, 0 or 2 – even)



What about XOR?

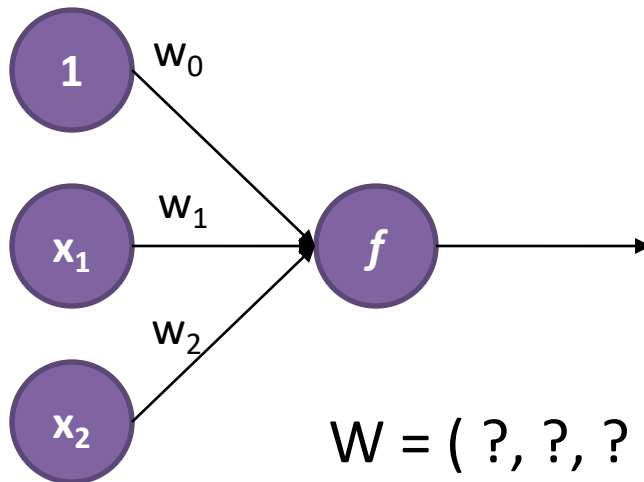
x_1	x_2	$x_1 \text{ XOR } x_2$
0	0	0
0	1	1
1	0	1
1	1	0



$$x_1 \oplus x_2$$

$w_0 = ?$

when $x_1 = x_2 = 0$
 $w_0 < 0$, say $w_0 = -50$

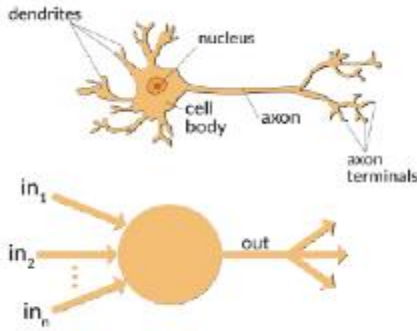


$w_1 = w_2 = w = ?$

when $x_1 = 0, x_2 = 1$
 or $x_1 = 1, x_2 = 0$
 $w + w_0 > 0$, say $w = 100$
 when $x_1 = 1, x_2 = 1$
 $w_1 + w_2 + w_0 < 0$
 ????

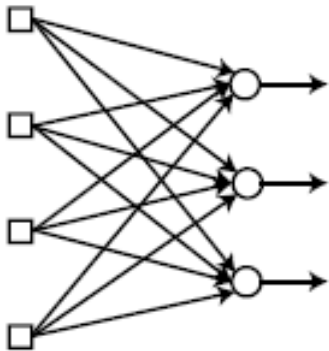
$W = (?, ?, ?)$

Historical Perspective



Perceptron

1954

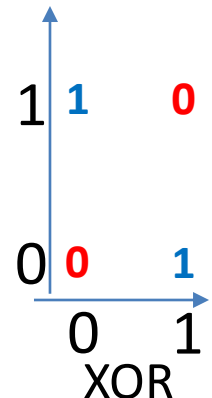
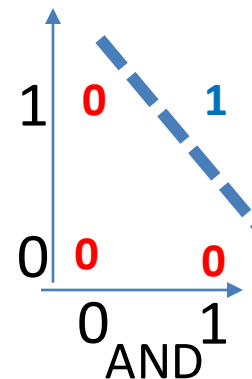
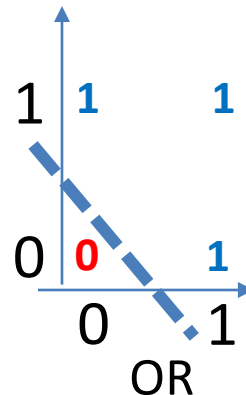


Single-layer Neural Network

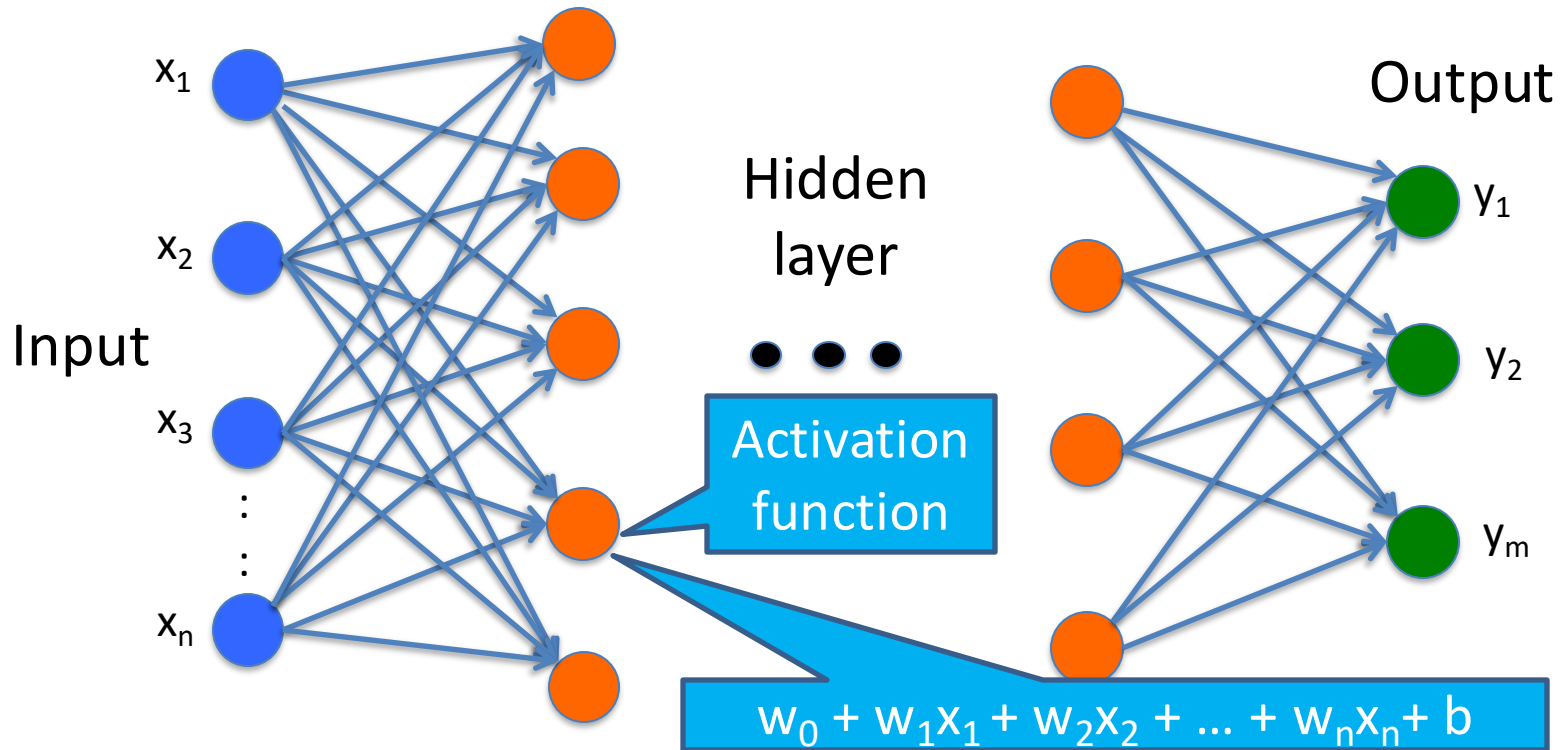
Quickly found it couldn't compute simple functions such as XOR (MIT book by Minsky)

Minsky and Papert (1969) showed that a single artificial neuron is incapable of implementing some simple functions such as the XOR logical function, parity, connectivity

Why can't perceptron compute XOR logical function?



Can Multilayer Perceptron Help?



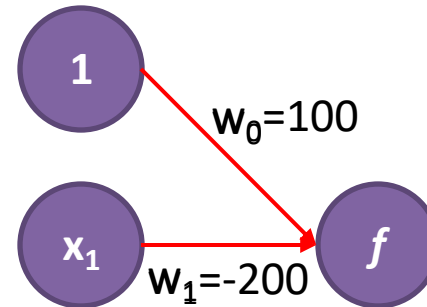
- Since each perceptron is a linear function, no matter how many perceptrons or layers, the final output remains linear.
- Hence multilayer neural network is not more powerful than a single perceptron in solving XOR (which is nonlinear).
- **BUT activation function is nonlinear.**

Is this argument valid?

Power of layering perceptrons

NOT

x_1	
0	1
1	0



$$W_{\text{NOT}} = (100, -200)$$

Logical complete with NOT, AND and OR
i.e., any logical function can be computed.

How to construct this function ?

$$(\bar{x}_1 \text{ AND } \bar{x}_2) \text{ OR } (x_1 \text{ AND } \bar{x}_2)$$

x_1	x_2	
0	0	1
0	1	0
1	0	1
1	1	0

Power of layering perceptrons

$(\bar{x}_1 \text{ AND } x_2) \text{ OR } (x_1 \text{ AND } \bar{x}_2)$

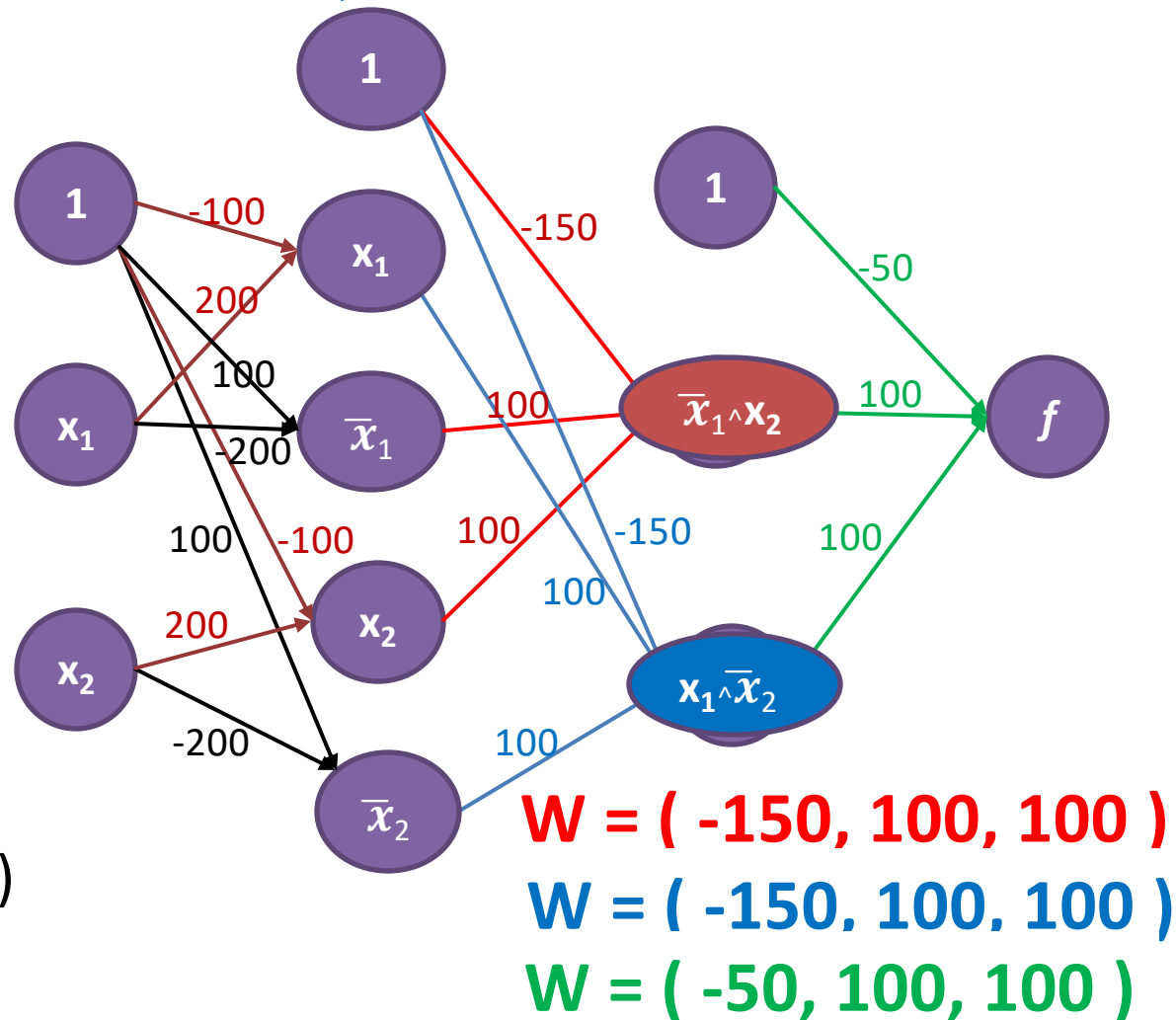
x_1	x_2	
0	0	0
0	1	1
1	0	1
1	1	0

$$W_{\text{NOT}} = (100, -200)$$

$$W_{\text{ID}} = (-100, 200)$$

$$W_{\text{AND}} = (-150, 100, 100)$$

$$W_{\text{OR}} = (-50, 100, 100)$$



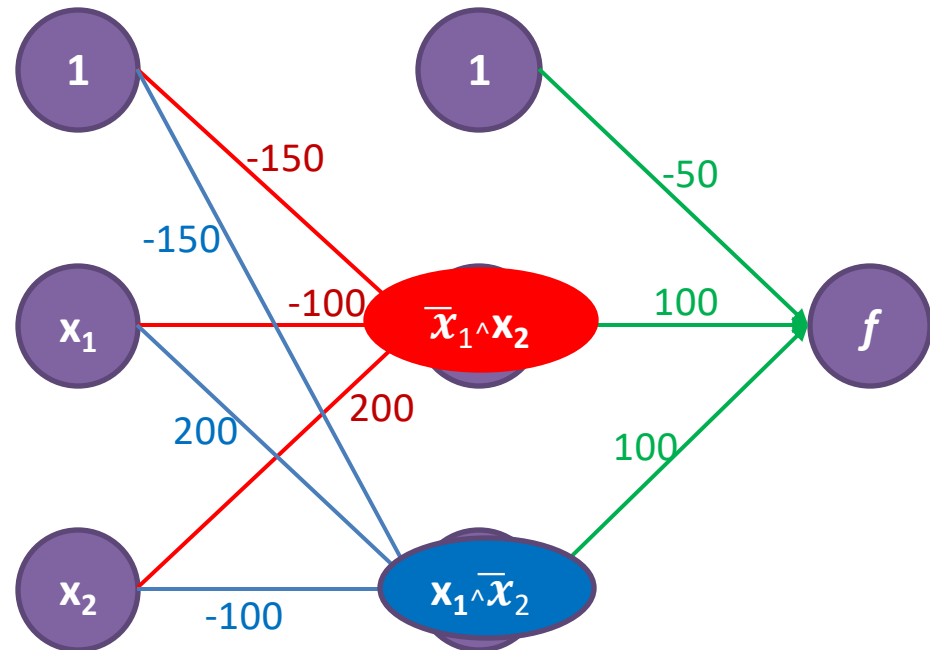
Power of layering perceptrons

$(\bar{x}_1 \text{ AND } x_2) \text{ OR } (x_1 \text{ AND } \bar{x}_2)$

x_1	x_2	
0	0	0
0	1	1
1	0	1
1	1	0

$$W_{\text{AND}} = (-150, 100, 100)$$

$$W_{\text{OR}} = (-50, 100, 100)$$



$$W = (-150, -100, 200)$$

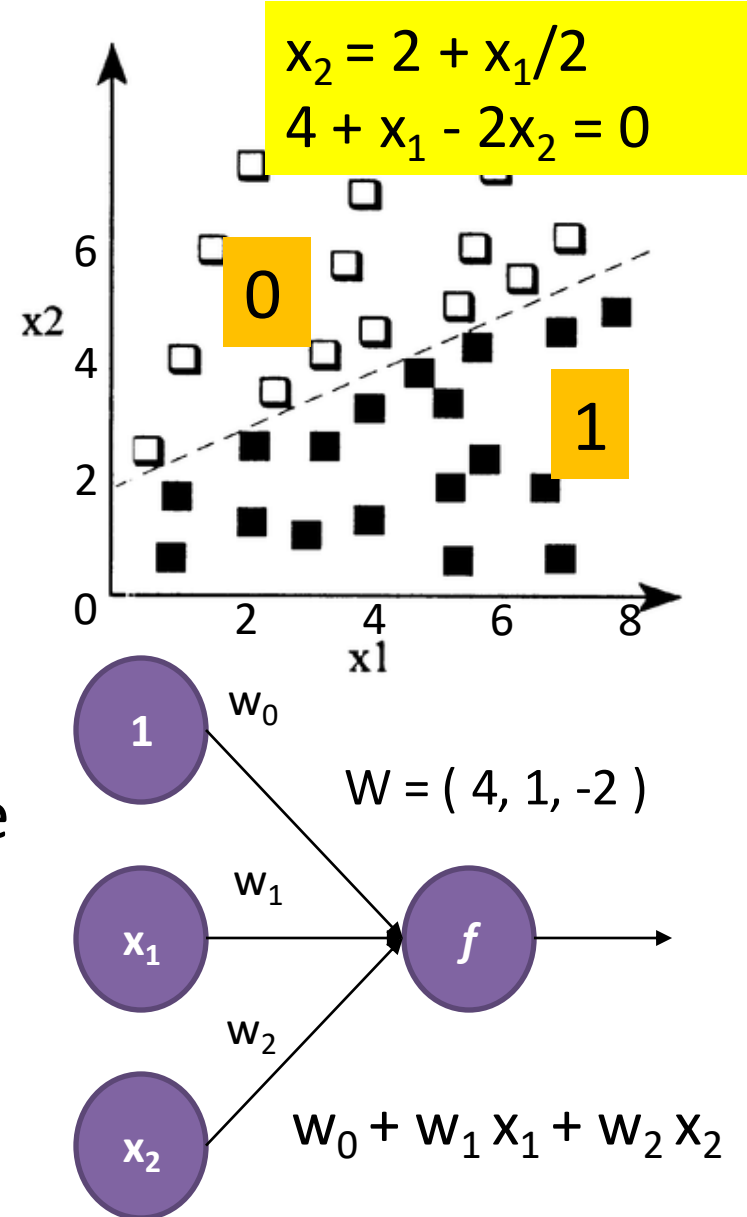
$$W = (-150, 200, -100)$$

$$W = (-50, 100, 100)$$

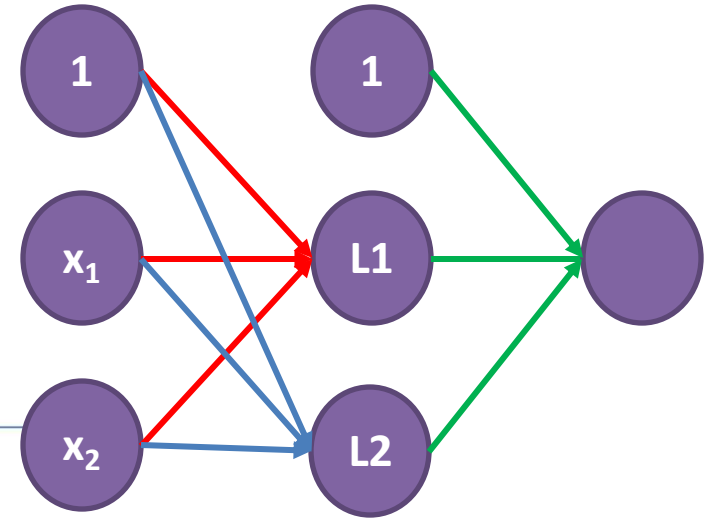
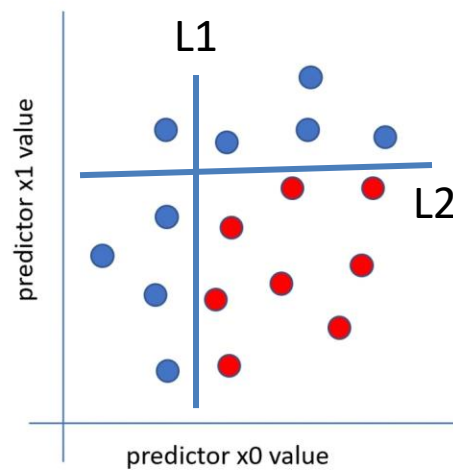
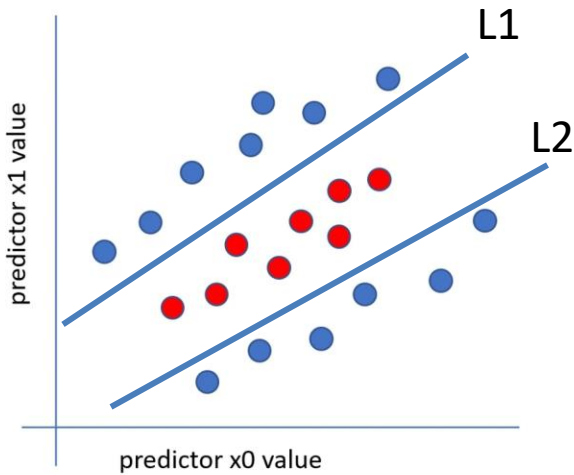
Linearly-Separable Classification

- **Linearly-separable** – there's a hyperplane (line if 2D) that can split input points into two classes.
- Classification problem can be solved by NN
- In general, the weights of NN corresponding to the hyperplane

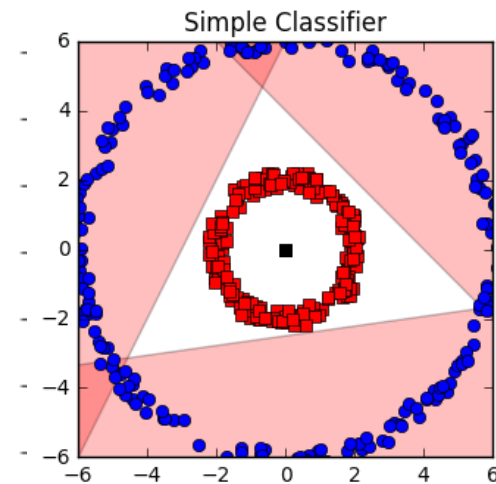
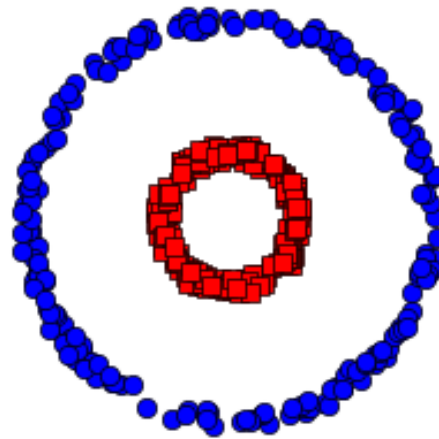
$$w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n$$



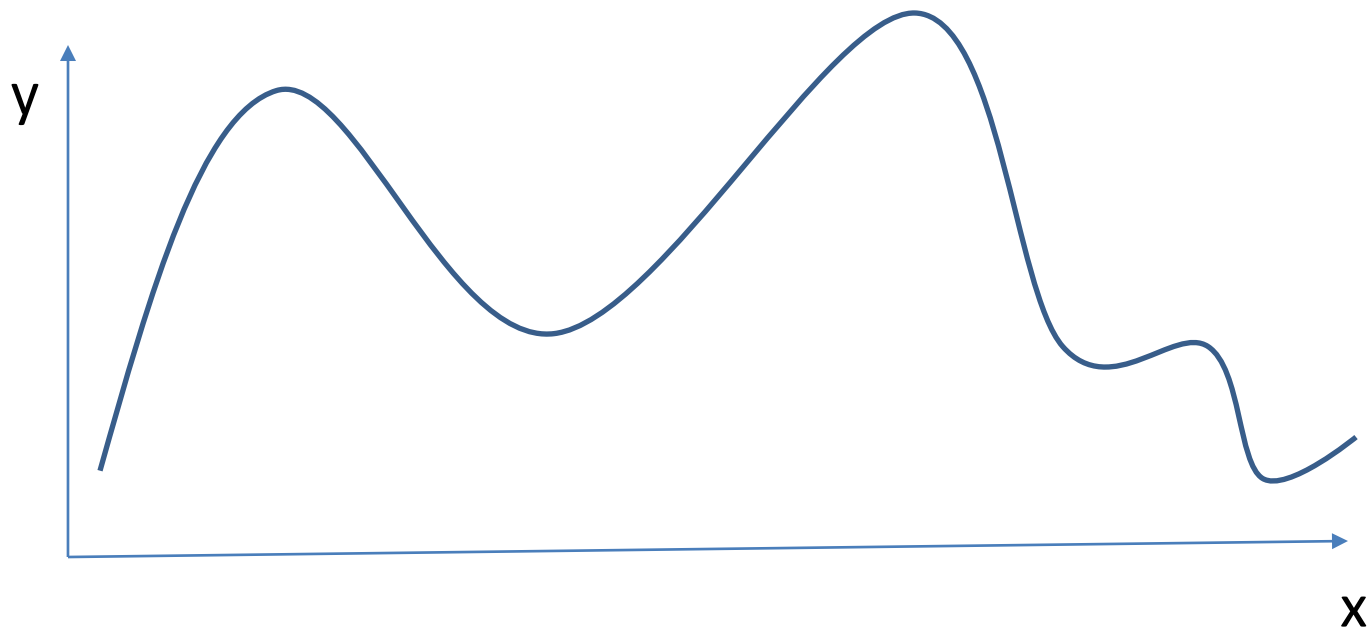
Non-Linearly Separable Data



Non-linearly Separable classification problems can be solved by NN with more layers using “AND”, “OR”, “NOT” functionality



5. NN as a Function Approximator for Regression

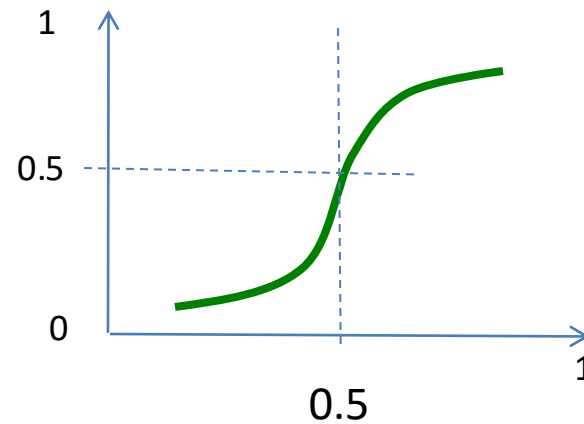
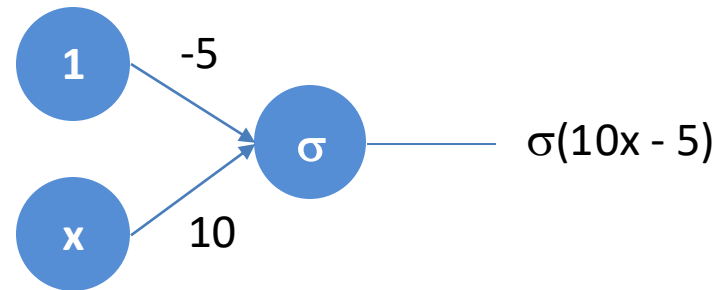
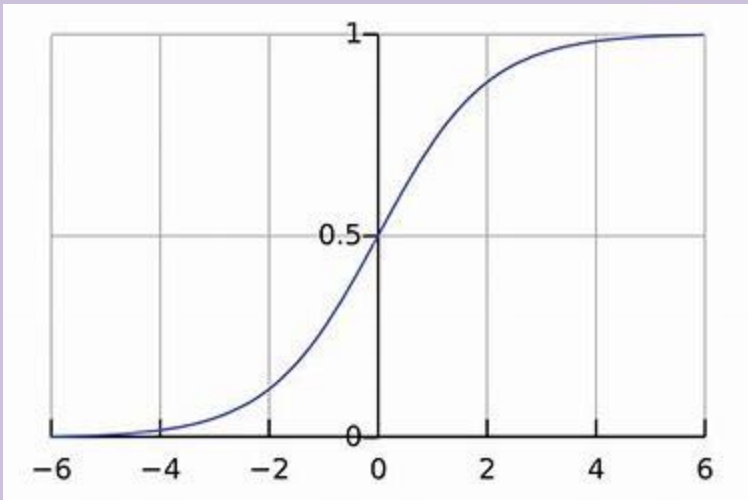


WLOG: Assume x is between $[0,1]$

NN as a Function Approximator

Consider **sigmoid** as activation function

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

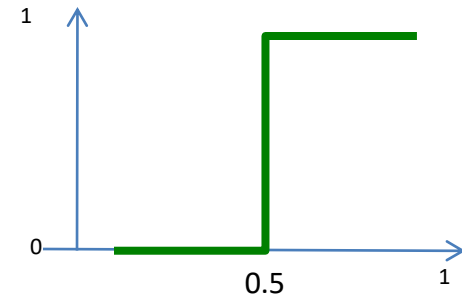
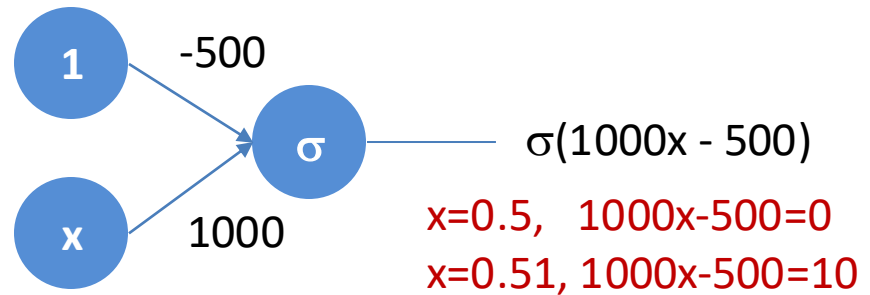
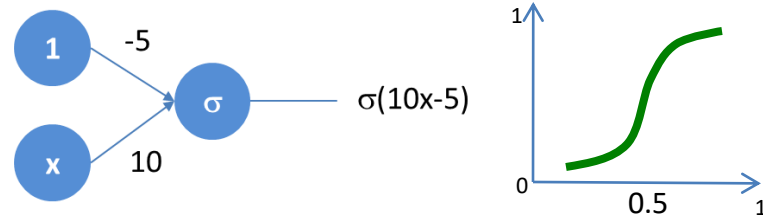
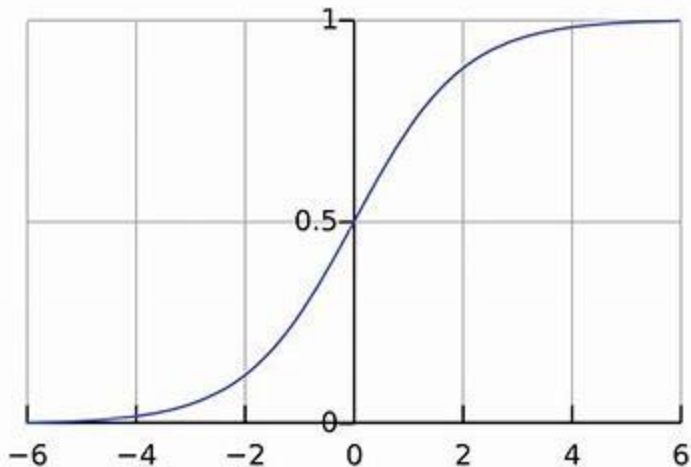


Assuming $0 \leq x \leq 1$

NN as a Function Approximator

Consider **sigmoid** as activation function

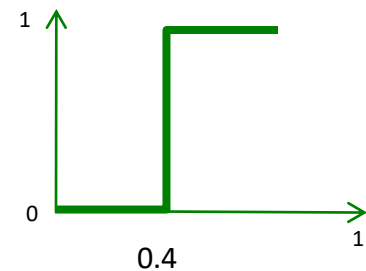
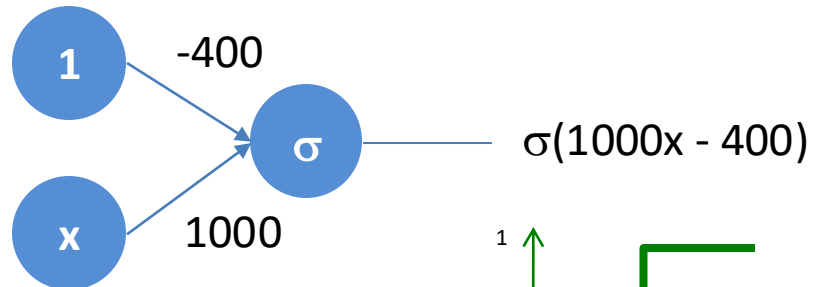
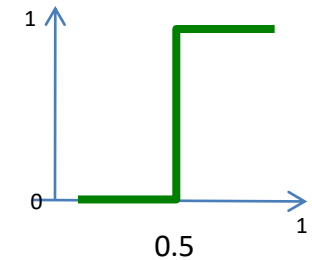
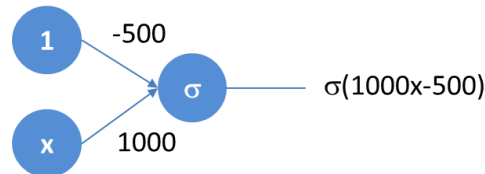
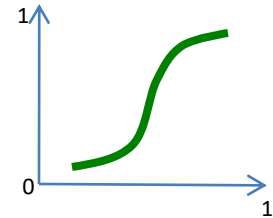
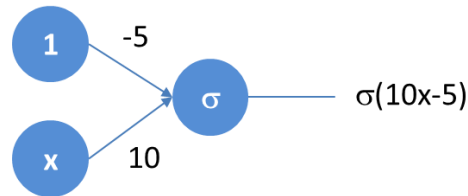
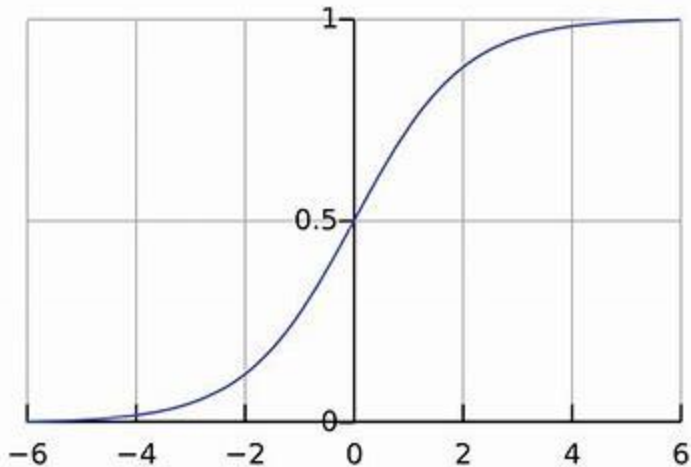
$$\sigma(x) = \frac{1}{1 + e^{-x}}$$



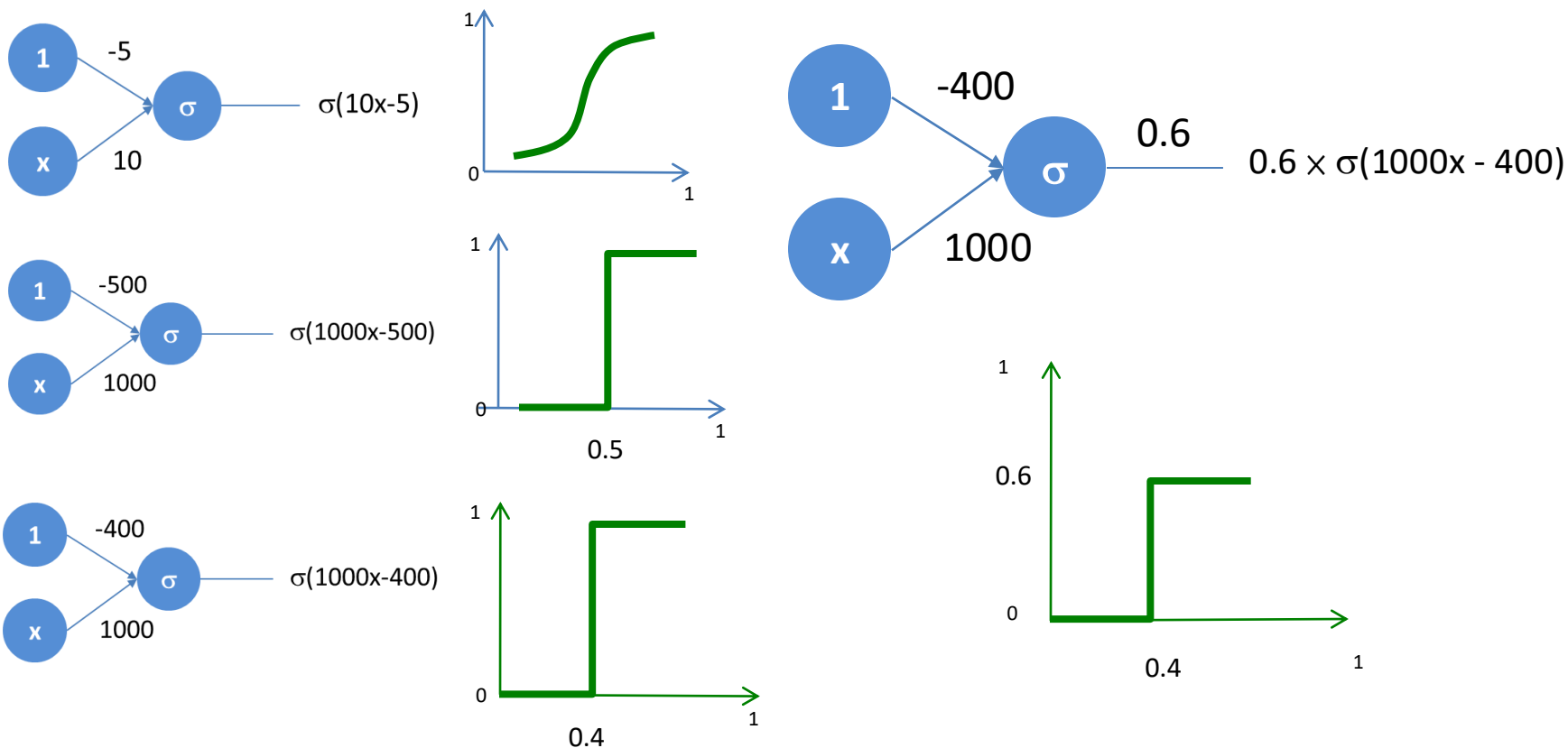
NN as a Function Approximator

Consider **sigmoid** as activation function

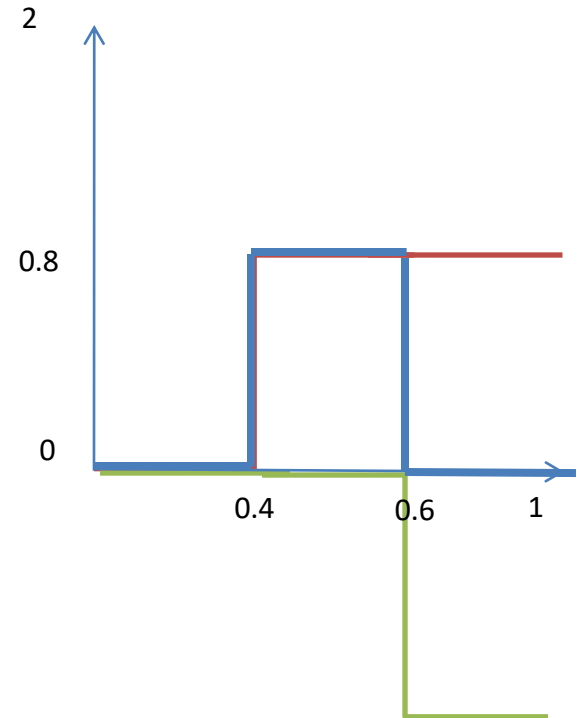
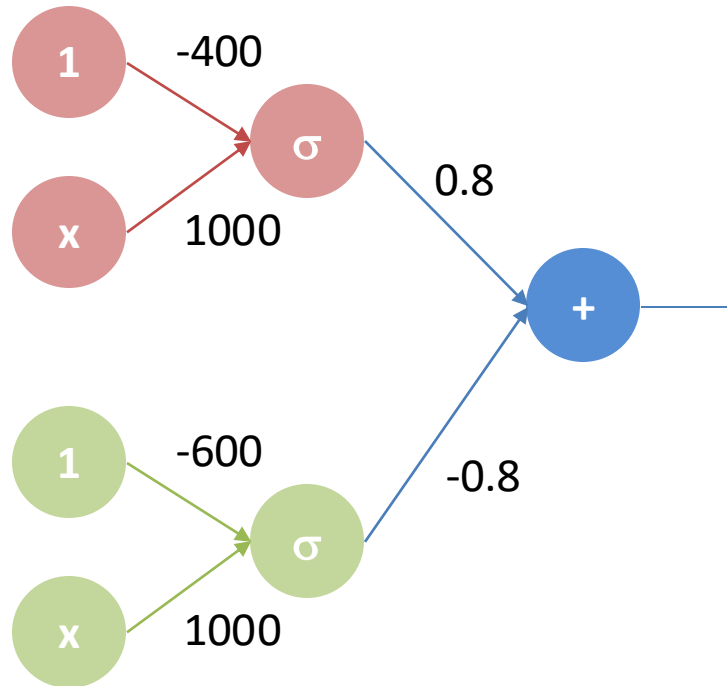
$$\sigma(x) = \frac{1}{1 + e^{-x}}$$



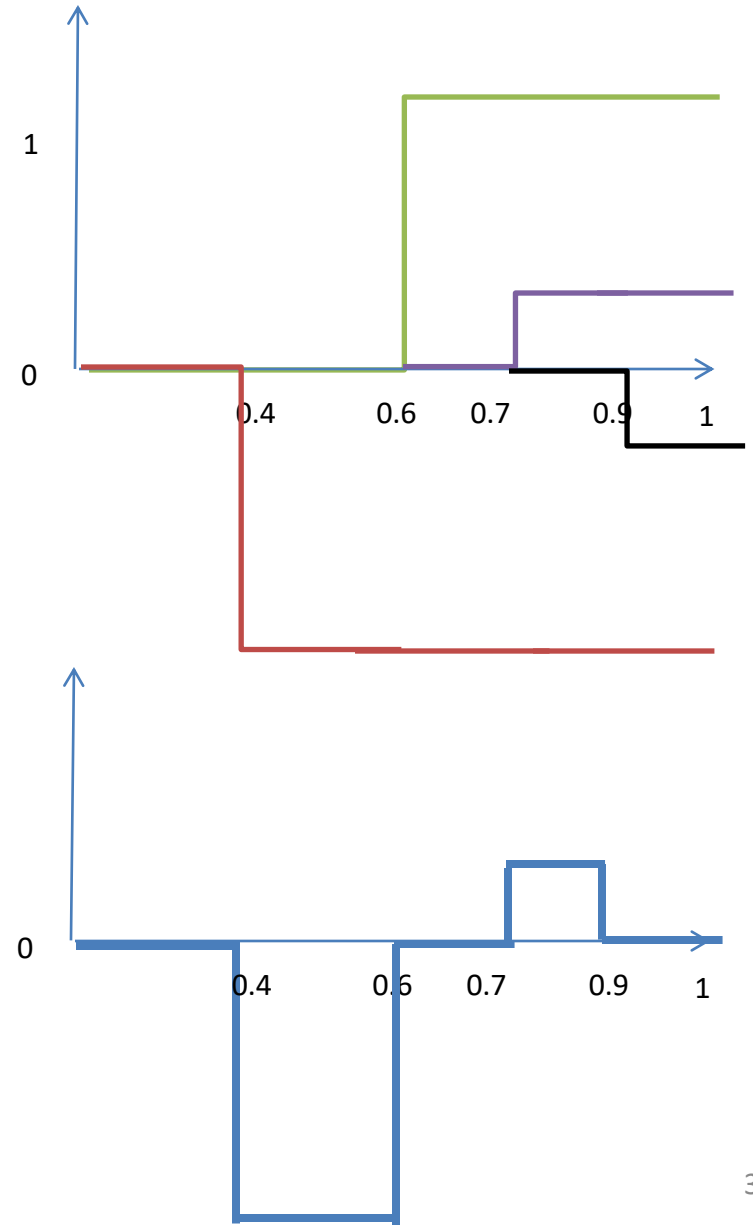
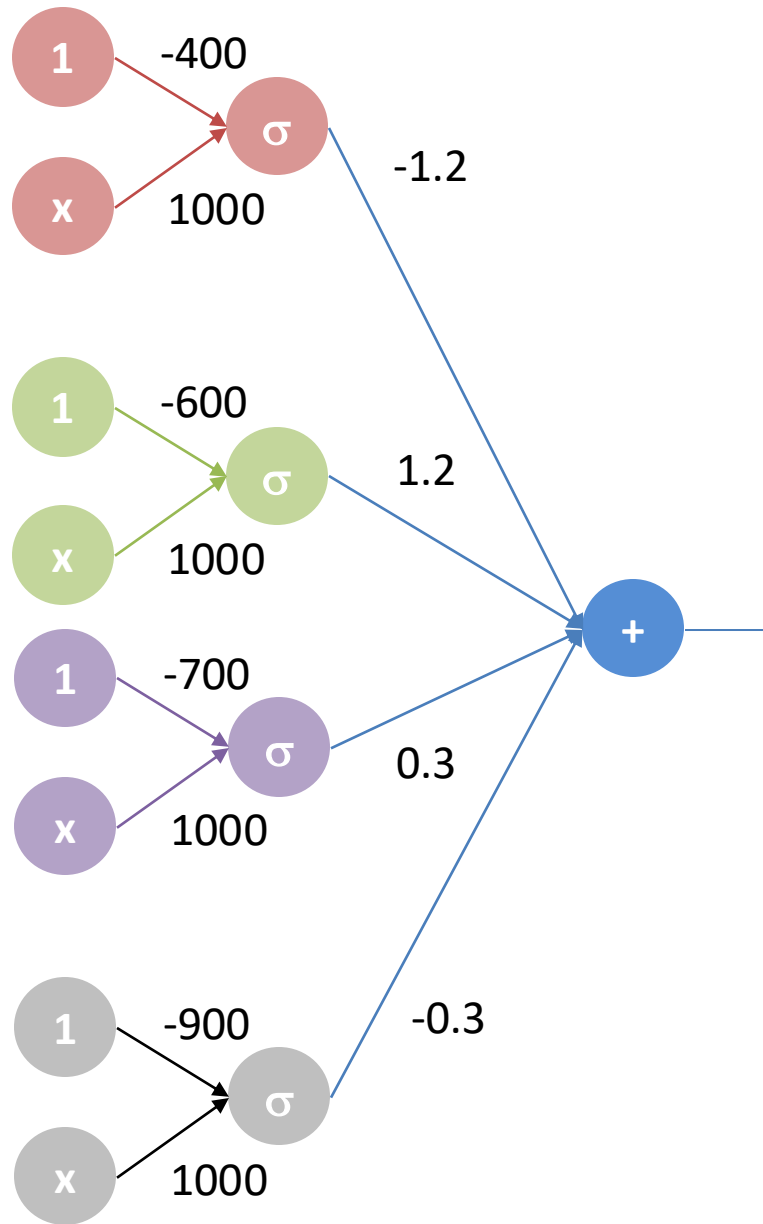
NN as a Function Approximator



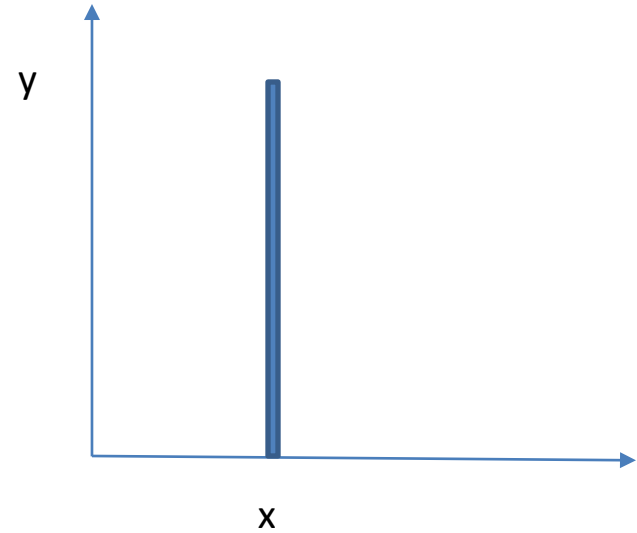
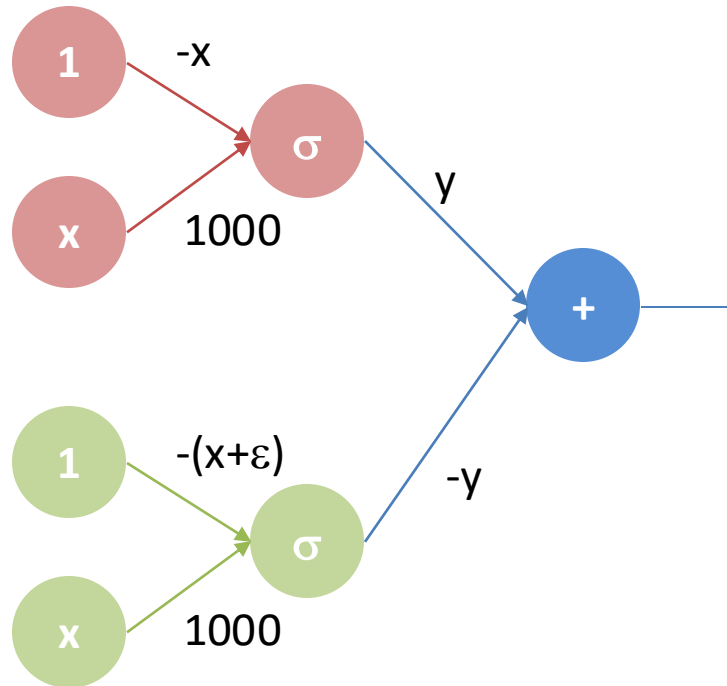
NN as a Function Approximator



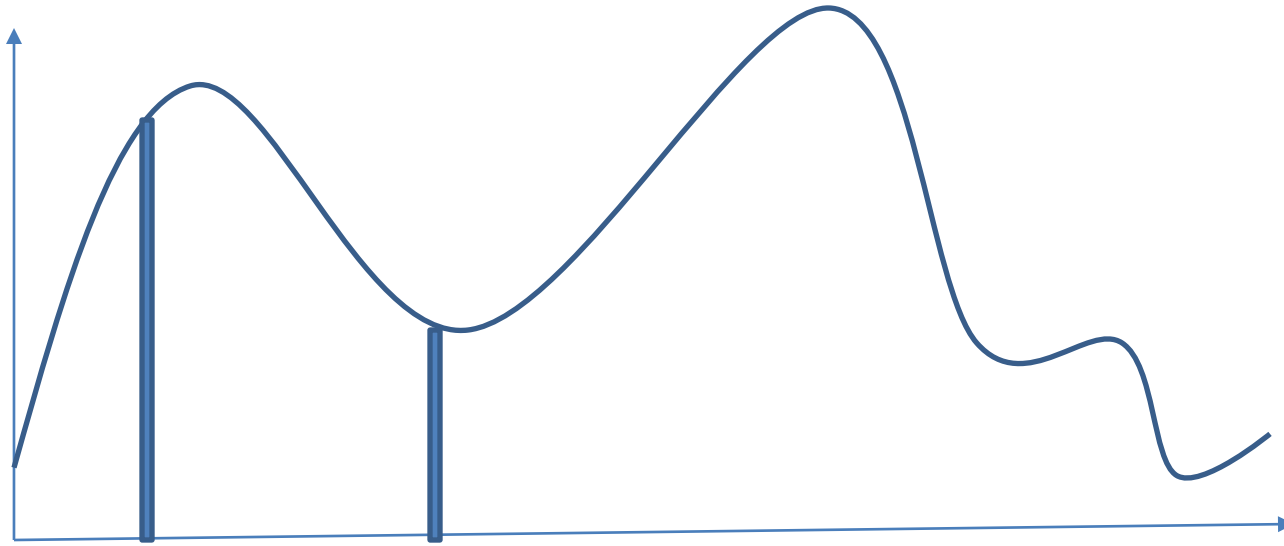
NN as a Function Approximator



NN as a Function Approximator



NN as a Function Approximator



NN with sufficient complexity can approximate any measurable function to any desired degree of accuracy. Sufficient capacity? Can be quite shallow.

6. Backpropagation (BP)

The screenshot shows the top of a Nature journal article page. At the top is a dark red header with a 'MENU' button and the 'nature' logo. Below this is a light blue bar with a color-coded icon, 'Altmetric: 75', 'Citations: 6342', and a 'More detail' link. The article title 'Learning representations by back-propagating errors' is prominently displayed, with a blue callout box pointing to the author 'Geoffrey Hinton, 2024 Nobel prize winner'. Below the title is the full author list: 'David E. Rumelhart, Geoffrey E. Hinton & Ronald J. Williams'. To the left of the article details, it says 'Letter'. On the right, there are two purple callout boxes: one stating '1986 paper in Nature with 44,000+ citations!!!' and another stating 'One of the main contributors for the success of Deep Learning (ABCD)'. At the bottom, there is a grey 'Abstract' section.

MENU ▾ nature
International journal of science

Altmetric: 75 Citations: 6342 [More detail >>](#)

Letter

Learning representations by back-propagating errors

Geoffrey Hinton, 2024 Nobel prize winner

David E. Rumelhart, Geoffrey E. Hinton & Ronald J. Williams

Nature **323**, 533–536 (09 October 1986)
doi:10.1038/323533a0
[Download Citation](#)

Received: 01 May 1986
Accepted: 31 July 1986
Published online: 09 October 1986

Abstract

We describe a new learning procedure, back-propagation, for networks of neurone-like units. The procedure repeatedly adjusts the weights of the connections in the network so as to minimize a measure of the difference between the actual output vector of the net and the desired output vector. As a result of the weight adjustments, internal 'hidden' units which are not part of the input or output come to represent important features of the task domain, and the regularities in the task

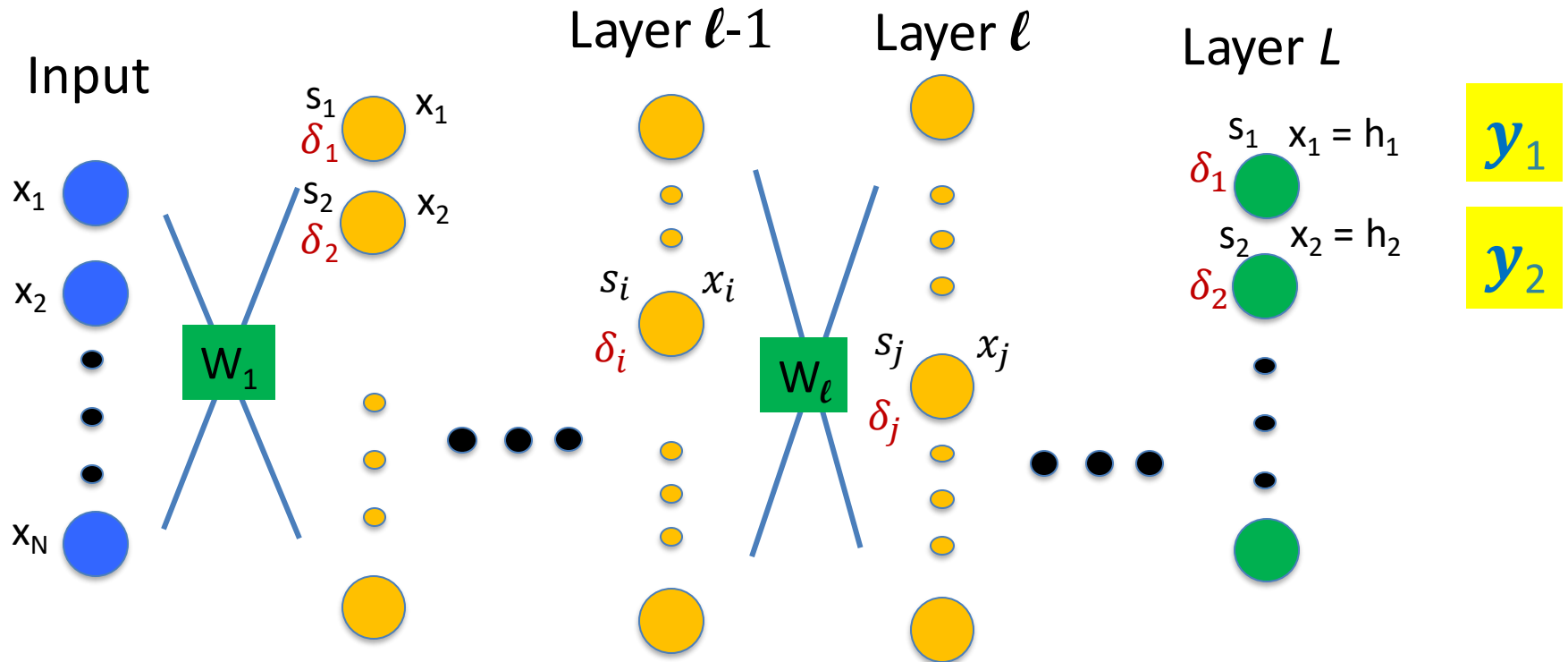
1986 paper
in *Nature*
with 44,000+
citations!!!

One of the main
contributors for
the success of
Deep Learning
(ABCD)

Backpropagation (BP)

- **Training** = adjusts the weights of the NN so that error (loss) of predictions is minimized
- **Gradient descent** = smart way to adjust weights
- **BP** is an algorithm that efficiently computes the gradient $\frac{\partial J(\mathbf{W})}{\partial w_{ij}^{(l)}}$ of the loss function $J(\mathbf{W})$ with respect to each weight $w_{ij}^{(l)}$ of the NN between neuron i and neuron j at layer l with $b_j^{(l)} = w_{0j}^{(l)}$

Backpropagation: going backward

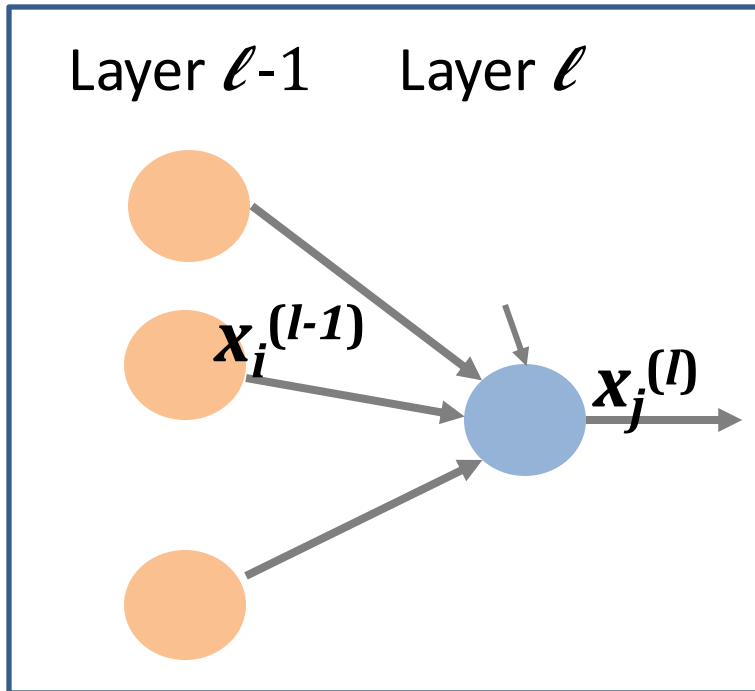


$$s_j^{(l)} = (\sum_i w_{ij}^{(l)} x_i^{(l-1)}) + b_j^{(l)}$$

$$x_j^{(l)} = g(s_j^{(l)})$$

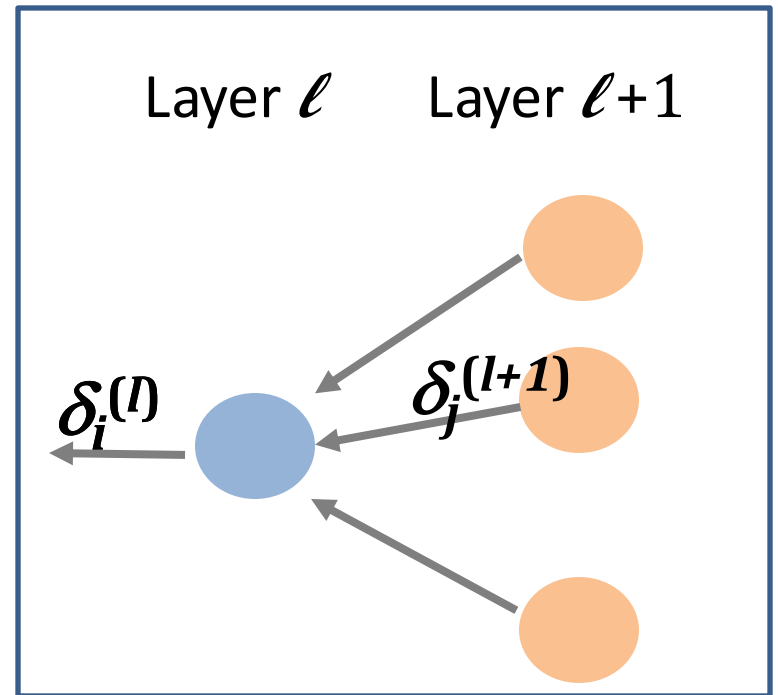
Compute $\delta_i^{(l-1)}$ in terms of $\delta_j^{(l)}$

Backpropagation: equation for δ



$$s_j^{(l)} = (\sum_i w_{ij}^{(l)} x_i^{(l-1)}) + b_j^{(l)}$$

$$x_j^{(l)} = g(s_j^{(l)})$$



$$\delta_i^{(l)} = g'(s_i^{(l)}) (\sum_j w_{ij}^{(l+1)} \delta_j^{(l+1)})$$

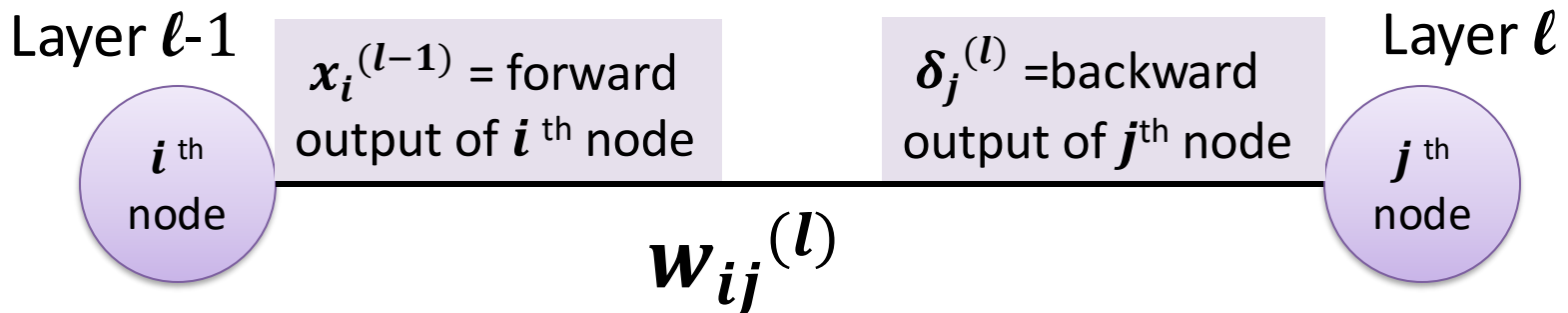
For output layer L :

$$\delta_i^{(L)} = g'(s_i^{(L)}) \left(\frac{\partial J}{\partial h(x)} \right)$$

7. Framework for Training the NN

1. Initialize all weights $w_{ij}^{(l)}$ at random [Note: Let $w_{0j}^{(l)} = b_j^{(l)}$]
2. Repeat :
3. Feed example(s) into layer 0
4. *Forward*: Compute all $x_j^{(l)}$ for $l = 1, \dots, L$
5. If error acceptable, go to step 8
6. *Backward*: Compute all $\delta_j^{(l)}$ for $l = L, \dots, 1$
7. Update weights: $w_{ij}^{(l)} \rightarrow w_{ij}^{(l)} - \alpha x_i^{(l-1)} \delta_j^{(l)}$
8. Return the final weights $w_{ij}^{(l)}$

$$\frac{\partial J(\mathbf{W})}{\partial w_{ij}^{(l)}}$$



Backpropagation: proof

GOAL: Want to show: $\frac{\partial J(\mathbf{w})}{\partial w_{ij}^{(l)}} = \delta_j^{(l)} \times x_i^{(l-1)}$

By induction (next few slides)

$$\frac{\partial J(\mathbf{w})}{\partial w_{ij}^{(l)}} = \frac{\partial J(\mathbf{w})}{\partial s_j^{(l)}} \times \frac{\partial s_j^{(l)}}{\partial w_{ij}^{(l)}} = \frac{\partial J(\mathbf{w})}{\partial s_j^{(l)}} \times x_i^{(l-1)}$$

$$s_j^{(l)} = \left(\sum_i w_{ij}^{(l)} x_i^{(l-1)} \right) + b_j^{(l)}$$

$$\Rightarrow \frac{\partial s_j^{(l)}}{\partial w_{ij}^{(l)}} = x_i^{(l-1)}$$

GOAL now becomes: want to show that $\delta_j^{(l)} = \frac{\partial J(\mathbf{w})}{\partial s_j^{(l)}}$

Backpropagation: proof

NEW GOAL : want to show that $\delta_j^{(l)} = \frac{\partial J(\mathbf{w})}{\partial s_j^{(l)}}$

$$\begin{aligned}\frac{\partial J(\mathbf{w})}{\partial s_i^{(l)}} &= \sum_j \frac{\partial J(\mathbf{w})}{\partial s_j^{(l+1)}} \times \frac{\partial s_j^{(l+1)}}{\partial x_i^{(l)}} \times \frac{\partial x_i^{(l)}}{\partial s_i^{(l)}} \\ &= \sum_j \delta_j^{(l+1)} \times \frac{\partial s_j^{(l+1)}}{\partial x_i^{(l)}} \times \frac{\partial x_i^{(l)}}{\partial s_i^{(l)}}\end{aligned}$$

By Induction (Backward)

with $\delta_j^{(k)} = \frac{\partial J(\mathbf{w})}{\partial s_j^{(k)}}$

for $k = L, L-1, \dots, l+1$

PROOF - SKIPPED

Backpropagation: proof

NEW GOAL : want to show that $\delta_j^{(l)} = \frac{\partial J(\mathbf{w})}{\partial s_j^{(l)}}$

$$\frac{\partial J(\mathbf{w})}{\partial s_i^{(l)}} = \sum_j \frac{\partial J(\mathbf{w})}{\partial s_j^{(l+1)}} \times \frac{\partial s_j^{(l+1)}}{\partial x_i^{(l)}} \times \frac{\partial x_i^{(l)}}{\partial s_i^{(l)}}$$

$$= \sum_j \delta_j^{(l+1)} \times \frac{\partial s_j^{(l+1)}}{\partial x_i^{(l)}} \times \frac{\partial x_i^{(l)}}{\partial s_i^{(l)}}$$

$$s_j^{(l+1)} = (\sum_i w_{ij}^{(l+1)} x_i^{(l)}) + b_j^{(l+1)}$$

$$\Rightarrow \frac{\partial s_j^{(l+1)}}{\partial x_i^{(l)}} = w_{ij}^{(l+1)}$$

Backpropagation: proof

NEW GOAL : want to show that $\delta_j^{(l)} = \frac{\partial J(\mathbf{w})}{\partial s_j^{(l)}}$

$$\begin{aligned}\frac{\partial J(\mathbf{w})}{\partial s_i^{(l)}} &= \sum_j \frac{\partial J(\mathbf{w})}{\partial s_j^{(l+1)}} \times \frac{\partial s_j^{(l+1)}}{\partial x_i^{(l)}} \times \frac{\partial x_i^{(l)}}{\partial s_i^{(l)}} \\&= \sum_j \delta_j^{(l+1)} \times \frac{\partial s_j^{(l+1)}}{\partial x_i^{(l)}} \times \frac{\partial x_i^{(l)}}{\partial s_i^{(l)}} \\&= \sum_j \delta_j^{(l+1)} \times w_{ij}^{(l+1)} \times \frac{\partial x_i^{(l)}}{\partial s_i^{(l)}} \\&= \sum_j \delta_j^{(l+1)} \times w_{ij}^{(l+1)} \times g'(s_i^{(l)}) \quad [\text{since } x_i^{(l)} = g(s_i^{(l)})]\end{aligned}$$

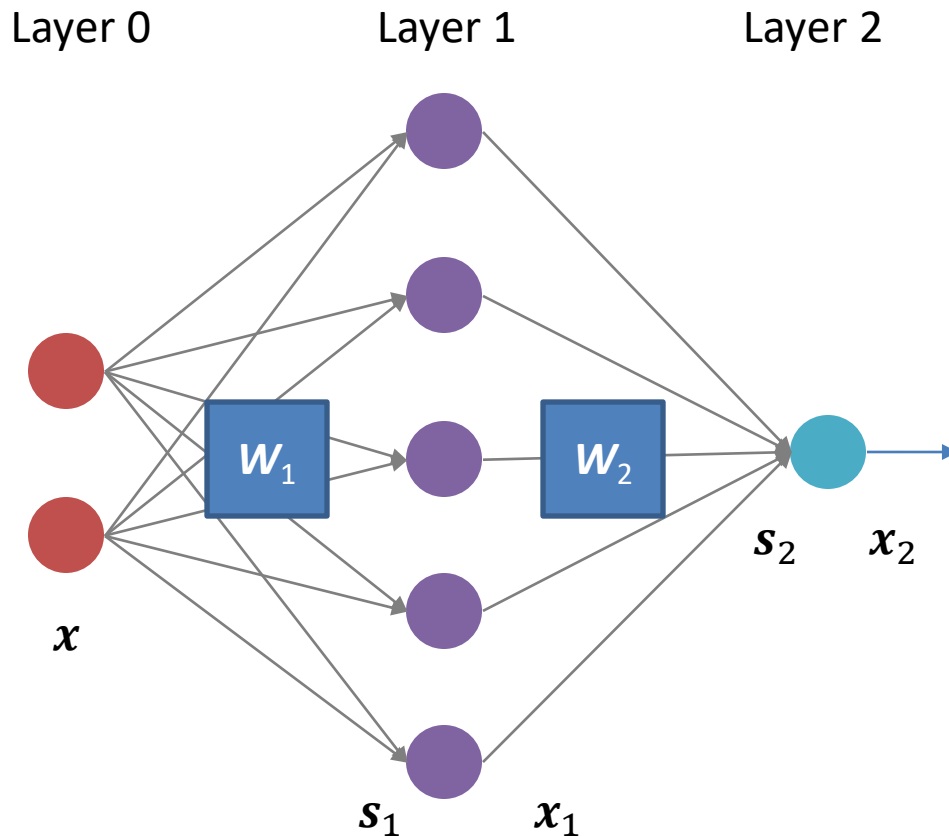
Backpropagation: proof

NEW GOAL : want to show that $\delta_j^{(l)} = \frac{\partial J(\mathbf{w})}{\partial s_j^{(l)}}$

$$\begin{aligned}\frac{\partial J(\mathbf{w})}{\partial s_i^{(l)}} &= \sum_j \frac{\partial J(\mathbf{w})}{\partial s_j^{(l+1)}} \times \frac{\partial s_j^{(l+1)}}{\partial x_i^{(l)}} \times \frac{\partial x_i^{(l)}}{\partial s_i^{(l)}} \\&= \sum_j \delta_j^{(l+1)} \times \frac{\partial s_j^{(l+1)}}{\partial x_i^{(l)}} \times \frac{\partial x_i^{(l)}}{\partial s_i^{(l)}} \\&= \sum_j \delta_j^{(l+1)} \times w_{ij}^{(l+1)} \times \frac{\partial x_i^{(l)}}{\partial s_i^{(l)}} \\&= \sum_j \delta_j^{(l+1)} \times w_{ij}^{(l+1)} \times g'(s_i^{(l)}) = \delta_i^{(l)}\end{aligned}$$

Recall: $\delta_i^{(l)} = g'(s_i^{(l)}) (\sum_j w_{ij}^{(l+1)} \delta_j^{(l+1)})$

8. Example of Simple NN Classifier



Forward pass equations:

$$s_1 = xW_1 + b_1$$

$$x_1 = \sigma(s_1)$$

$$s_2 = x_1W_2 + b_2$$

$$h = x_2 = \sigma(s_2)$$

Dimensions of matrices:

$$x: 1 \times 2$$

$$W_1: 2 \times 5$$

$$b_1: 1 \times 5$$

$$s_1, x_1: 1 \times 5$$

$$W_2: 5 \times 1$$

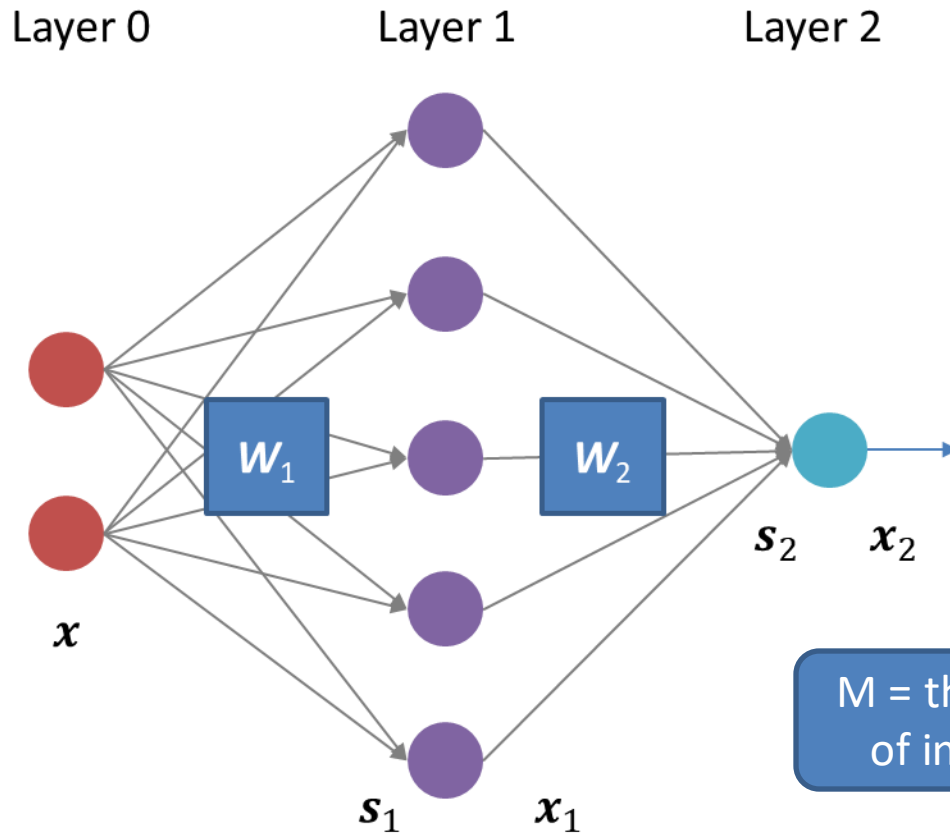
$$b_2: 1 \times 1$$

$$s_2, x_2: 1 \times 1$$

Let x_i is the forward output of layer i (here x_1, x_2)

Let δ_i is the backward output of layer i (here δ_1, δ_2)

Example of Simple NN Classifier



Forward pass equations:

$$s_1 = xW_1 + b_1$$

$$x_1 = \sigma(s_1)$$

$$s_2 = x_1W_2 + b_2$$

$$h = x_2 = \sigma(s_2)$$

Dimensions of matrices:

$$x: \mathbf{M} \times 2$$

$$W_1: 2 \times 5$$

$$b_1: \mathbf{M} \times 5$$

$$s_1, x_1: \mathbf{M} \times 5$$

$$W_2: 5 \times 1$$

$$b_2: \mathbf{M} \times 1$$

$$s_2, x_2: \mathbf{M} \times 1$$

M = the number
of input data

Let x_i is the forward output of layer i (here x_1, x_2)

Let δ_i is the backward output of layer i (here δ_1, δ_2)

Example of Simple NN Classifier

Calculating δ_2

In general, for output layer L :

$$\delta_i^{(L)} = g'(s_i^{(L)}) \left(\frac{\partial J}{\partial h(x)} \right)$$

For the example, output layer 2:

$$\delta_2 = \sigma'(s_2) \left(\frac{\partial J}{\partial h} \right)$$

Loss function for binary classification is binary cross entropy:

$$J = -y \log h - (1 - y) \log(1 - h)$$

Applying the general formula $\frac{\partial}{\partial x} \log x = \frac{1}{x}$:

$$\frac{\partial J}{\partial h} = \frac{-y}{h} + \frac{1-y}{1-h} = \frac{-y+h-y+h-hy}{h(1-h)} = \frac{h-y}{h(1-h)}$$

As for $\sigma'(s_2)$:

$$\sigma(s) = \frac{1}{1+e^{-s}} \rightarrow \sigma'(s) = \frac{e^{-s}}{(1+e^{-s})^2} = \left(\frac{1+e^{-s}-1}{1+e^{-s}} \right) \left(\frac{1}{1+e^{-s}} \right) = (1 - \sigma(s))\sigma(s)$$

Since $h = x_2 = \sigma(s_2)$, we have $\sigma'(s_2) = (1 - h)h$

$$\delta_2 = \sigma'(s_2) \left(\frac{\partial J}{\partial h} \right) = (1 - h)h \frac{h-y}{h(1-h)} = h - y$$

Example of Simple NN Classifier

	Loss Function J	Activation Function g	$\delta^{(l)}$
Binary Classification	Binary cross entropy	Sigmoid	$h(x) - y$
Multi Classification	Cross entropy	Softmax	$\mathbf{h}(x) - \mathbf{y}$

Calculating δ_1

In general, for layer l :

$$\delta_i^{(l)} = g'(s_i^{(l)}) \left(\sum w_{ij}^{(l+1)} \delta_j^{(l+1)} \right)$$

For the example, recall matrix dimensions:

$$s_2, x_2: \quad M \times 1 \quad \delta_2$$

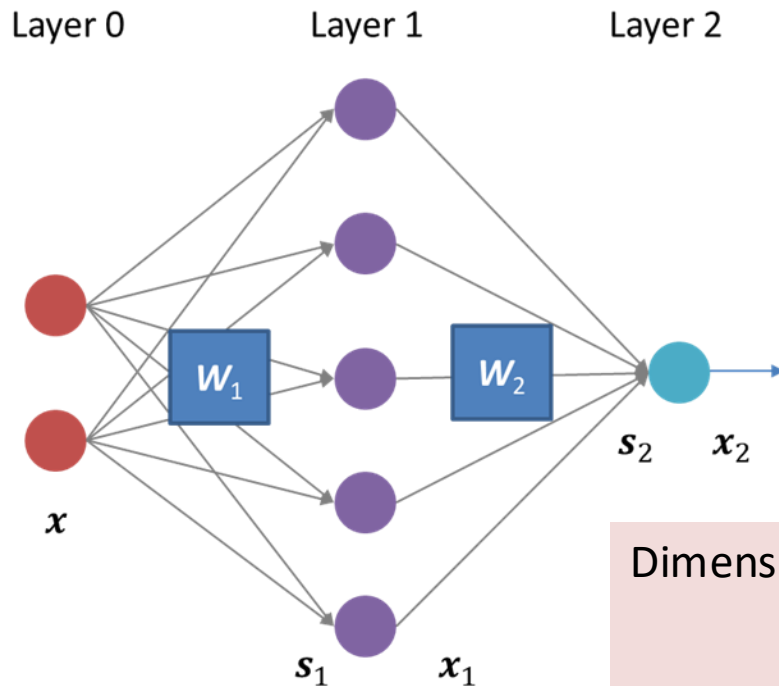
$$W_2 \quad : \quad 5 \times 1$$

$$s_1, x_1: \quad M \times 5 \quad \delta_1$$

For layer 1:

$$\delta_1 = \sigma'(s_1) \circ \delta_2 W_2^T = (1 - \sigma(s_1)) \sigma(s_1) \circ \delta_2 W_2^T$$

Example of Simple NN Classifier



Forward pass equations:

$$s_1 = xW_1 + b_1$$

$$x_1 = \sigma(s_1)$$

$$s_2 = x_1W_2 + b_2$$

$$h = x_2 = \sigma(s_2)$$

Backward pass equations:

$$\delta_2 = h - y$$

$$\delta_1 = (1 - \sigma(s_1)) \sigma(s_1) \circ \delta_2 W_2^T$$

Dimensions of matrices:

$$x: M \times 2$$

$$W_1: 2 \times 5$$

$$b_1: M \times 5$$

$$\delta_1, s_1, x_1: M \times 5$$

$$W_2: 5 \times 1$$

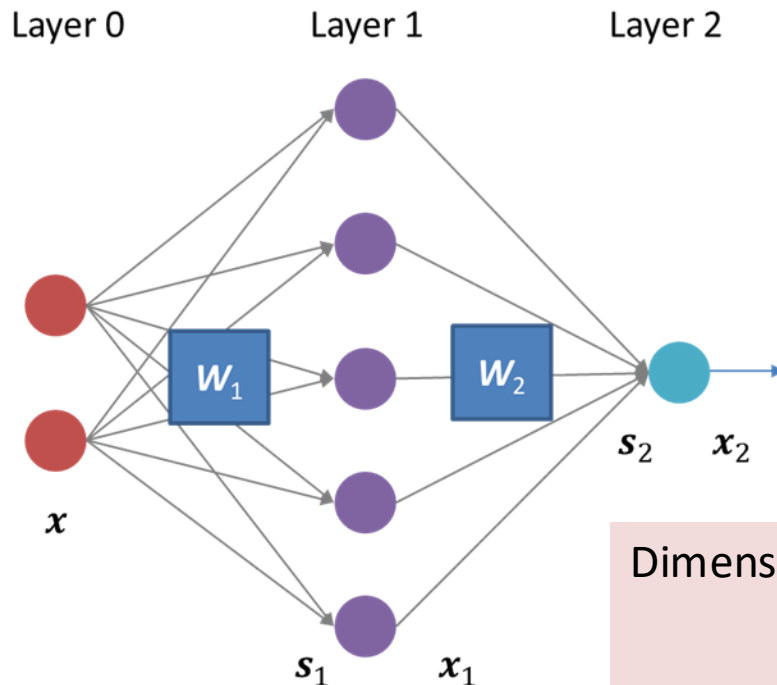
$$b_2: M \times 1$$

$$\delta_2, s_2, x_2: M \times 1$$

Let x_i is the forward output of layer i (here x_1, x_2)

Let δ_i is the backward output of layer i (here δ_1, δ_2)

Example of Simple NN Classifier



Dimensions of matrices:

$$\begin{aligned}
 x &: M \times 2 \\
 W_1 &: 2 \times 5 \\
 b_1 &: M \times 5 \\
 \delta_1, s_1, x_1 &: M \times 5 \\
 W_2 &: 5 \times 1 \\
 b_2 &: M \times 1 \\
 \delta_2, s_2, x_2 &: M \times 1
 \end{aligned}$$

Forward pass equations:

$$\begin{aligned}
 s_1 &= xW_1 + b_1 \\
 x_1 &= \sigma(s_1) \\
 s_2 &= x_1W_2 + b_2 \\
 h &= x_2 = \sigma(s_2)
 \end{aligned}$$

Backward pass equations:

$$\begin{aligned}
 \delta_2 &= h - y \\
 \delta_1 &= (1 - \sigma(s_1)) \sigma(s_1) \delta_2 W_2^T
 \end{aligned}$$

Update weights:

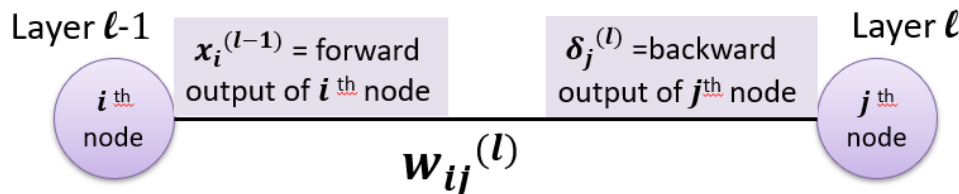
$$\begin{aligned}
 W_2 &= W_2 - \alpha x_1^T \delta_2 \\
 b_2 &= b_2 - \alpha \delta_2 \\
 W_1 &= W_1 - \alpha x^T \delta_1 \\
 b_1 &= b_1 - \alpha \delta_1
 \end{aligned}$$

Let x_i is the forward output of layer i (here x_1, x_2)
 Let δ_i is the backward output of layer i (here δ_1, δ_2)

Example of Simple NN Classifier

Framework for Training the NN

1. Initialize all weights $w_{ij}^{(l)}$ at random [Note: Let $w_{0j}^{(l)} = b_j^{(l)}$]
2. Repeat :
3. Feed example(s) into layer 0
4. **Forward**: Compute all $x_j^{(l)}$ for $l = 1, \dots, L$
5. If error acceptable, go to step 8
6. **Backward**: Compute all $\delta_j^{(l)}$ for $l = L, \dots, 1$
7. Update weights: $w_{ij}^{(l)} \rightarrow w_{ij}^{(l)} - \alpha x_i^{(l-1)} \delta_j^{(l)}$
8. Return the final weights $w_{ij}^{(l)}$



Forward pass equations:

$$\begin{aligned} s_1 &= xW_1 + b_1 \\ x_1 &= \sigma(s_1) \\ s_2 &= x_1W_2 + b_2 \\ h &= x_2 = \sigma(s_2) \end{aligned}$$

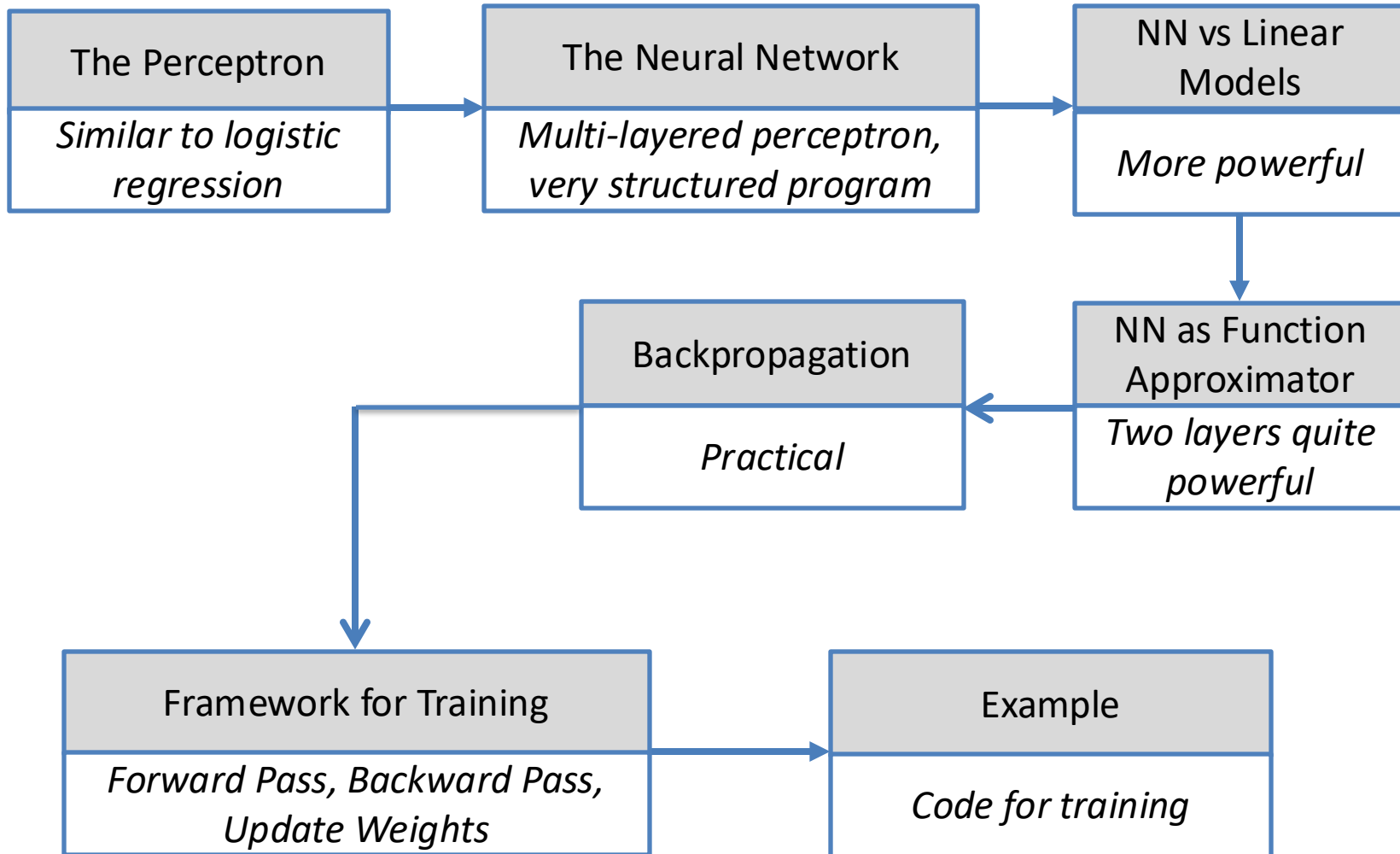
Backward pass equations:

$$\begin{aligned} \delta_2 &= h - y \\ \delta_1 &= (1 - \sigma(s_1)) \sigma(s_1) \delta_2 W_2^T \end{aligned}$$

Update weights:

$$\begin{aligned} W_2 &= W_2 - \alpha x_1^T \delta_2 \\ b_2 &= b_2 - \alpha \delta_2 \\ W_1 &= W_1 - \alpha x^T \delta_1 \\ b_1 &= b_1 - \alpha \delta_1 \end{aligned}$$

NN Basics – Mind Map



NN Training – Outline

1. Gradient Descent, SGD and Mini-batch SGD
2. Vanishing Gradient Problem
3. Not Zero-Centered Problem
4. Choice of Activation Function
5. Learning Rate Problems
6. Momentum and Nesterov Momentum
7. Adaptive Learning Rates
8. Overfitting Problem
9. Weight Regularization, Dropout, Early Stopping
10. Network Distillation
11. Batch Normalization

1. Gradient Descent (GD)

- Minimize $J(W) = \frac{1}{M} \sum_{i=1}^M e(h(\mathbf{x}_i), y_i)$
- By taking iterative steps along $-\nabla J(W)$:
$$\Delta W = -\alpha \nabla J(W)$$
- ∇J computed based on all M examples

Stochastic Gradient Descent (SGD)

- Minimize $J(W) = \frac{1}{M} \sum_{i=1}^M e(h(\mathbf{x}_i), y_i)$
- By taking iterative steps along $-\nabla J(W)$:
$$\Delta W = -\alpha \nabla J(W)$$
- ∇J computed based on all M examples
- Stochastic GD: randomly pick one $(\mathbf{x}_i, y_i)$
- Expected value of negative gradient of $(\mathbf{x}_i, y_i)$:
$$E[-\nabla e(h(\mathbf{x}_i), y_i)] = \frac{1}{M} \sum_{i=1}^M -\nabla e(h(\mathbf{x}_i), y_i) = -\nabla J$$

Alternatively: SGD can pick few samples.

Mini-batch SGD

- Minimize $J(W) = \frac{1}{B} \sum_{i=1}^B e(h(x_i), y_i)$
- By taking iterative steps along $-\nabla J(W)$:
$$\Delta W = -\alpha \nabla J(W)$$
- ∇J computed based on **all** B examples
- Instead of updating weights based on single random example as input, use a **batch** of random examples
- Advantages:
 - More accurate estimation of gradient
 - Smoother convergence
 - Mini-batches lead to fast training!
 - Can parallelize & achieve speed increases on GPUs

Mini-batch SGD

Framework for training NN

1. Initialize all weights $w_{ij}^{(l)}$ at random
2. Repeat :
3. Pick a **mini-batch** of (can be one) examples to feed into layer 0
4. **Forward**: Compute all $x_j^{(l)}$
5. If error acceptable, go to step 8
6. **Backward**: Compute all $\delta_j^{(l)}$
7. Update weights: $w_{ij}^{(l)} \rightarrow w_{ij}^{(l)} - \alpha x_i^{(l-1)} \delta_j^{(l)}$
8. Return the final weights $w_{ij}^{(l)}$

2. Vanishing Gradient Problem

Key equations for backpropagation: $\delta_i^{(l)}$

For output layer L : $\delta_i^{(L)} = g'(s_i^{(L)}) \left(\frac{\partial J}{\partial h(x)} \right)$

For earlier layers for $l = L-1, \dots, 1$:

$$\delta_i^{(l)} = g'(s_i^{(l)}) \left(\sum w_{ij}^{(l+1)} \delta_j^{(l+1)} \right)$$

$$= g'(s_i^{(l)}) \left(\sum w_{ij}^{(l+1)} g'(s_j^{(l+1)}) \left(\sum w_{jk}^{(l+2)} \delta_k^{(l+2)} \right) \right)$$

$$= g'(s_i^{(l)}) \left(\sum w_{ij}^{(l+1)} g'(s_j^{(l+1)}) \left(\sum w_{jk}^{(l+2)} \delta_k^{(l+2)} \right) \right)$$

$$= g'(\) \dots g'(\) \dots g'(\) \dots \quad [\text{up to } L \text{ occurrences}]$$

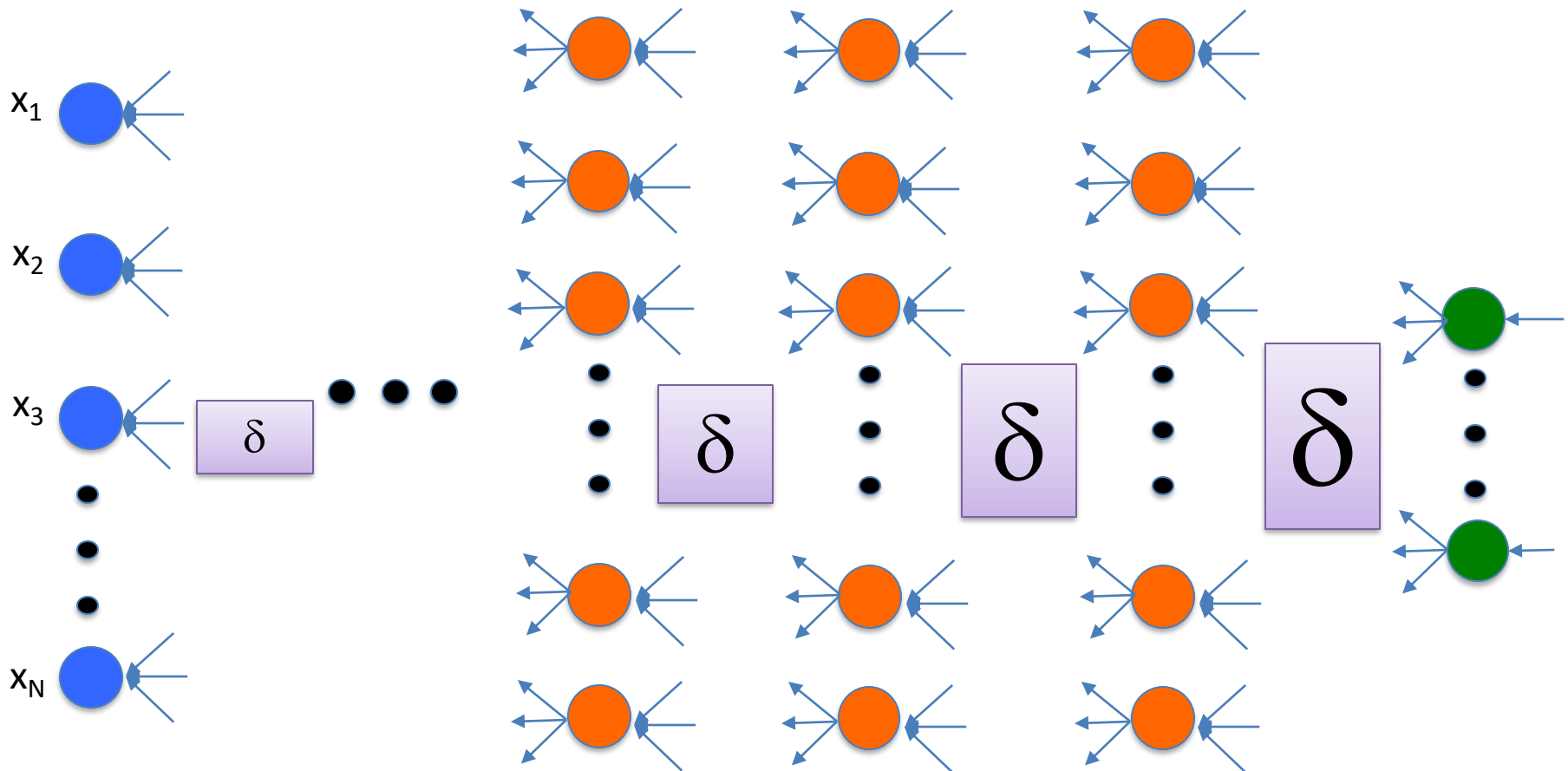
Vanishing Gradient Problem

When $g'()$ is small (e.g. $g = \text{Sigmoid}, g'() \leq 0.25$);

δ 's get smaller and smaller as you go back

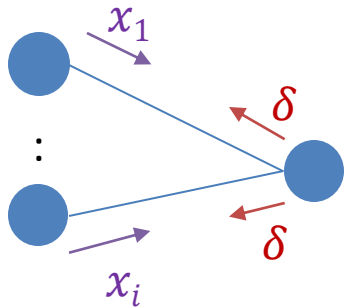
because $\delta = g'() \dots g'() \dots g'() \dots$

$\Rightarrow$ smaller changes to weights as you go back



3. Not Zero-Centered Problem

What happens when we have only positive values for $x = g(s)$ (i.e. g is sigmoid)?



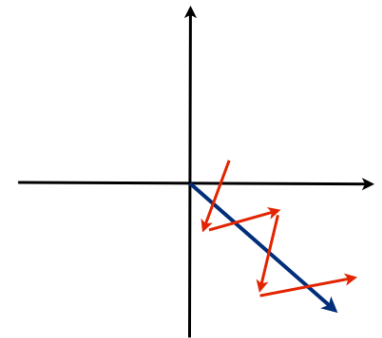
($x_1 \delta, \dots, x_i \delta, \dots$) will be either all positive or negative depending on δ

Suppose blue vector is optimal direction for gradient (assume a simple case with 2 weights).

Since the weights either all increase or all decrease, then we get a **zig-zag path**, not good for gradient descent.

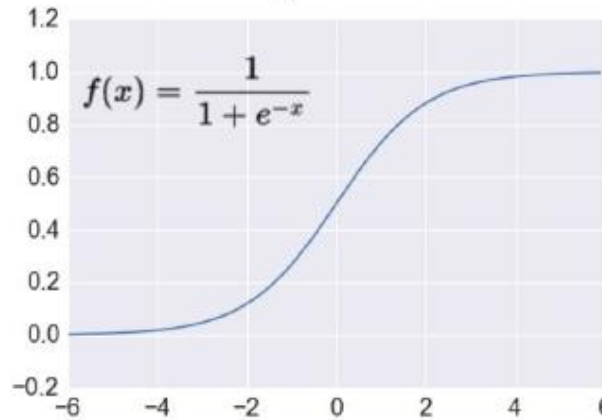
Similarly, non zero-centered activation functions are not ideal, they produce more positive or more negative output values.

Thus, we want activation function to be **zero-centered**.

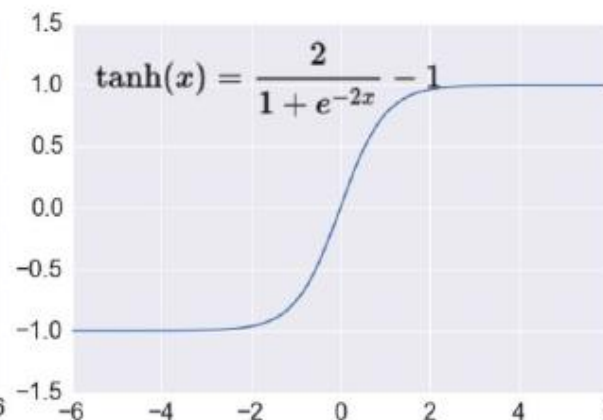


4. Choice of Activation Function

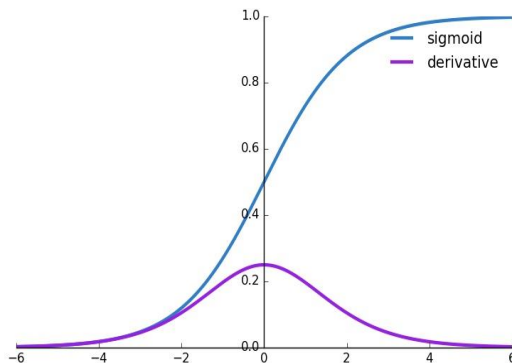
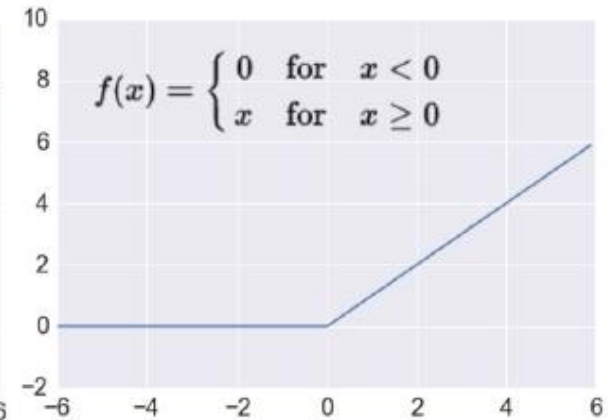
Sigmoid



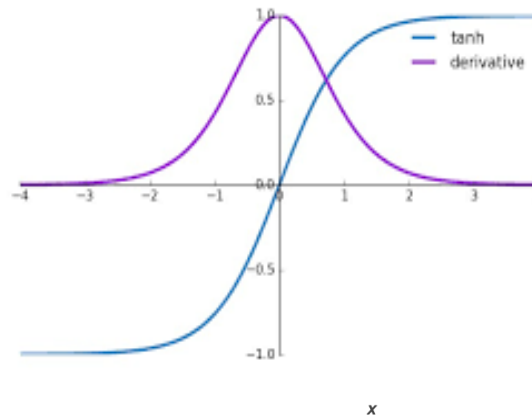
TanH



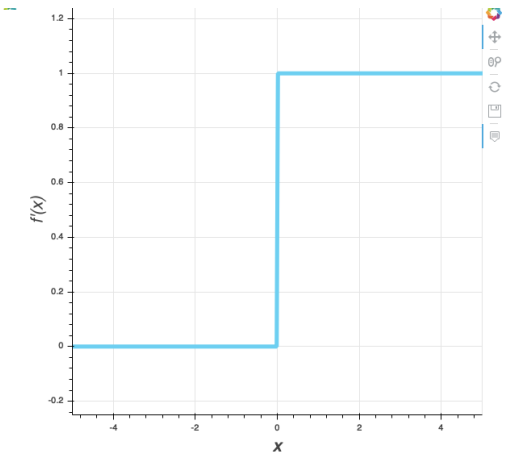
ReLU



$$g'(s) = (1 - g(s))g(s)$$



$$g'(s) = 1 - g(s)^2$$

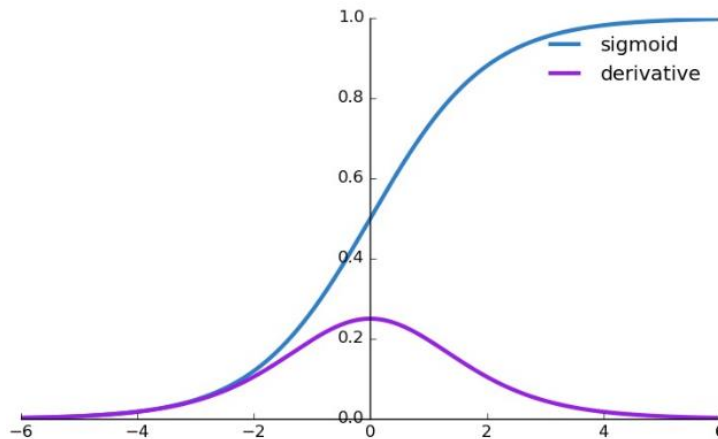


$$g'(s) = \begin{cases} 0 & \text{if } s < 0 \\ 1 & \text{if } s \geq 0 \end{cases}$$

Choice of Activation Function: sigmoid

$$g(s) = \sigma(s) = \frac{1}{1 + e^{-s}}$$

$$\rightarrow g'(s) = (1 - g(s))g(s)$$



- When s is very small or very large, $g'(s)$ is very small.
- Even when s isn't, $g'(s) \leq 0.25$.

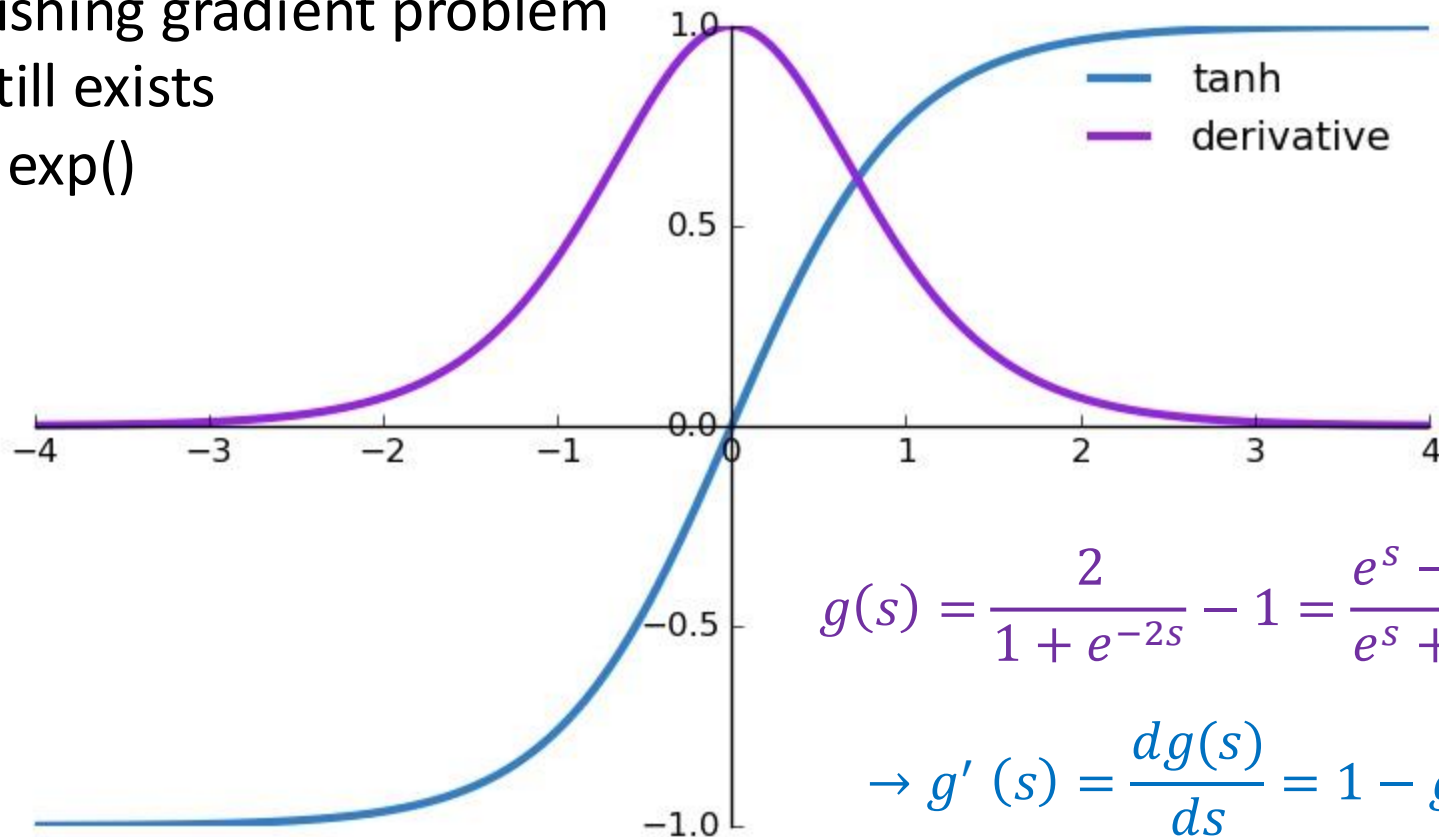
Historically popular because of nice interpretation of probability by squeezing numbers in $[0, 1]$ for classification.

Shortcomings:

- Vanishing gradient
- Sigmoid outputs are not zero-centered (mean > 0)
- $\exp()$ is expensive to compute

Choice of Activation Function: tanh

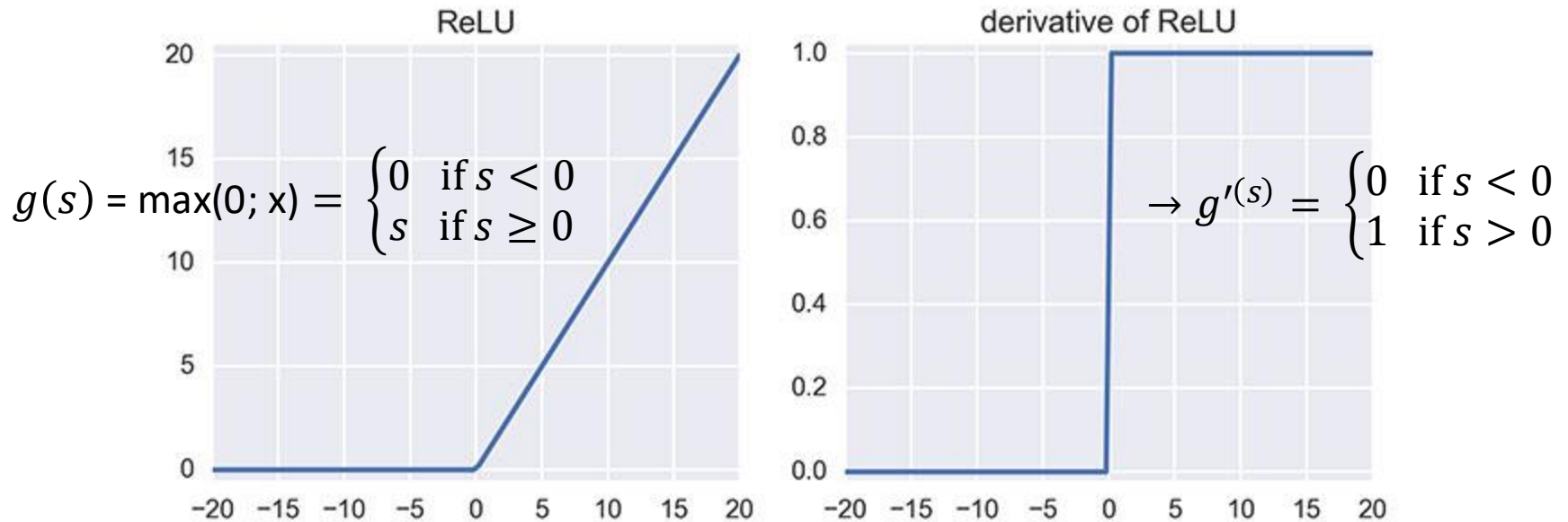
- Squashes numbers in $[-1, 1]$
- Zero centered
- Vanishing gradient problem still exists
- Still exp()



$$g(s) = \frac{2}{1 + e^{-2s}} - 1 = \frac{e^s - e^{-s}}{e^s + e^{-s}}$$

$$\rightarrow g'(s) = \frac{dg(s)}{ds} = 1 - g(s)^2$$

Choice of Activation Function: ReLU

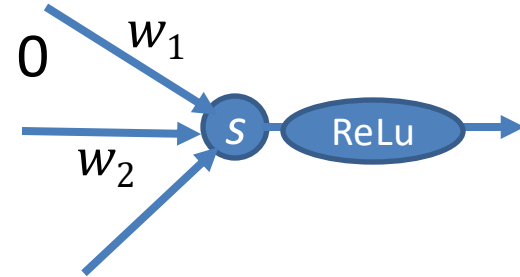


- No vanishing gradient problem when positive, but still exists when negative (gradient = 0 at 0)
- Converges much faster than sigmoid/tanh (a factor of 6)
- Computationally efficient
- Not zero-centered
- Widely used in CNN (since AlexNet) – to be learned later
- Seems to model better (neurons has the dropout effect)

Choice of Activation Function: ReLU

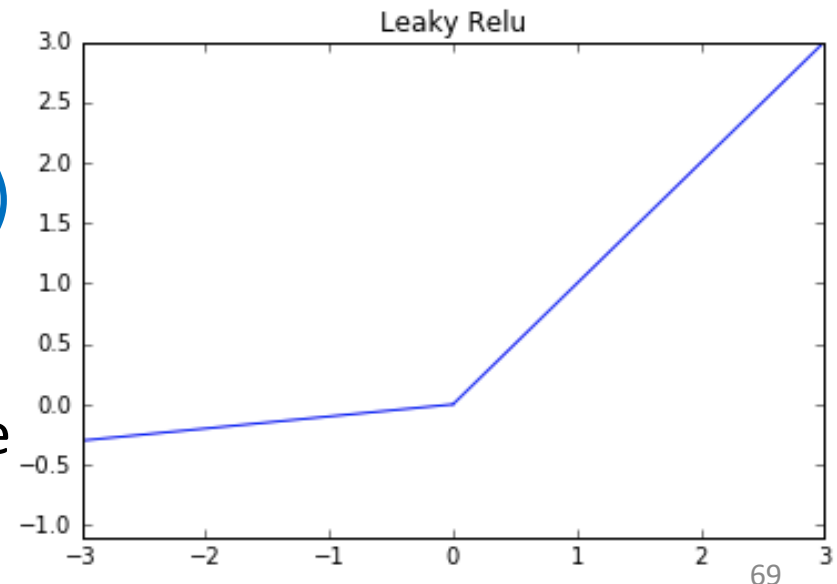
Problem for ReLU:

- Gradient is 0 for negative s
- Weights will not get adjusted when gradient is 0
- **Dying ReLU problem:** neurons will stop (die) simply because gradient is 0, nothing changes (s remains -ve and gradient remains 0).
- As much as 40% of the network can be “dead” (i.e., neurons never activate across the entire training dataset)



Leaky Relu: $f(x) = \max(0.01x, x)$

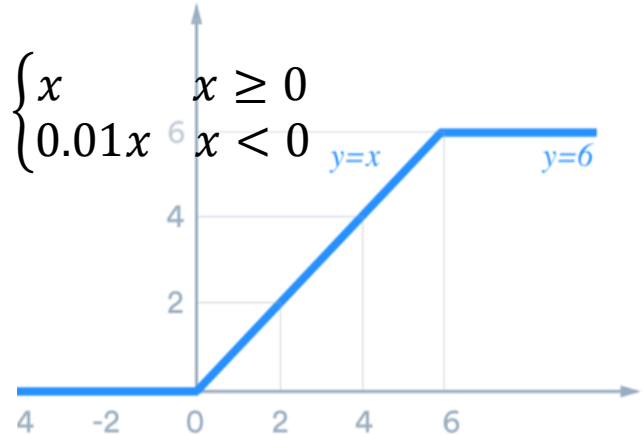
- All benefits of ReLU
- Closer to zero-centered outputs
- Non-zero gradient when negative



Choice of Activation Function: ReLU

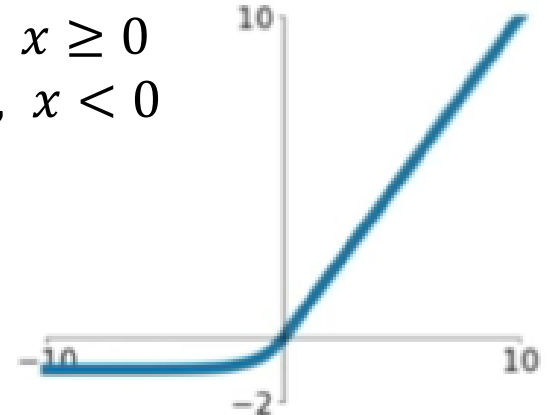
Variations in ReLU

- **Leaky ReLU** - $f(x) = \max(0.01x, x)$ $f(x) = \begin{cases} x & x \geq 0 \\ 0.01x & x < 0 \end{cases}$
- **Parametric ReLU** - $f(x) = \max(ax, x)$
- **ReLU6** - $f(x) = \min(\max(x, 0), 6)$
- clipping the gradient at some value n



Exponential Linear Units (ELU): $f(x) = \begin{cases} x, & x \geq 0 \\ \alpha(e^x - 1), & x < 0 \end{cases}$

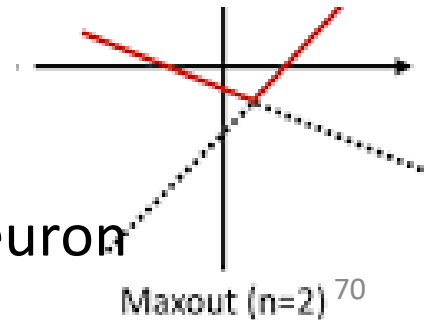
- Similar to leaky ReLU
- Differentiable at 0 when $\alpha=1$
- Drawback: Higher computation for $\exp()$



Maxout "Neuron"

$\text{Max}(w_1x + b_1, w_2x + b_2)$

- Generalizes ReLU and Leaky ReLU
- Drawback: doubles number of parameters per neuron



Maxout (n=2) ⁷⁰